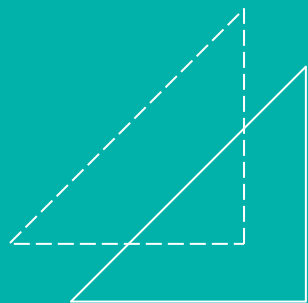


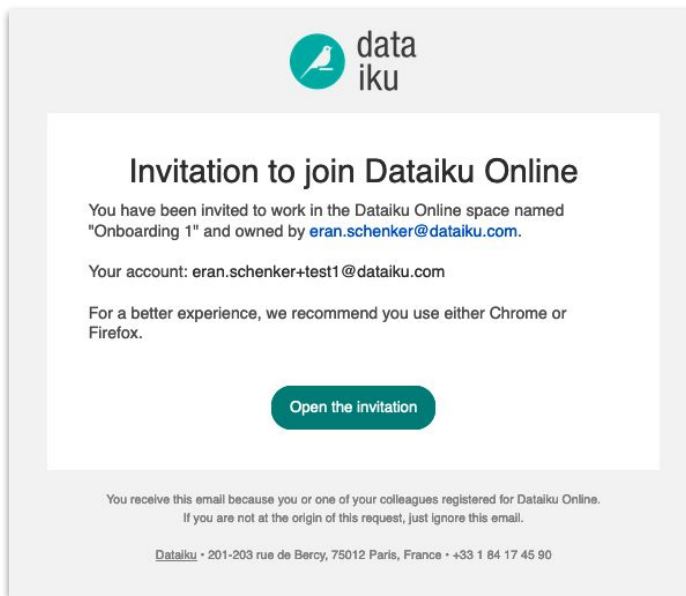


ADVANCED DESIGNER

Connect to the Instance and Set up the project

Log in to the training instance

You should have received an email invitation to join the training instance:

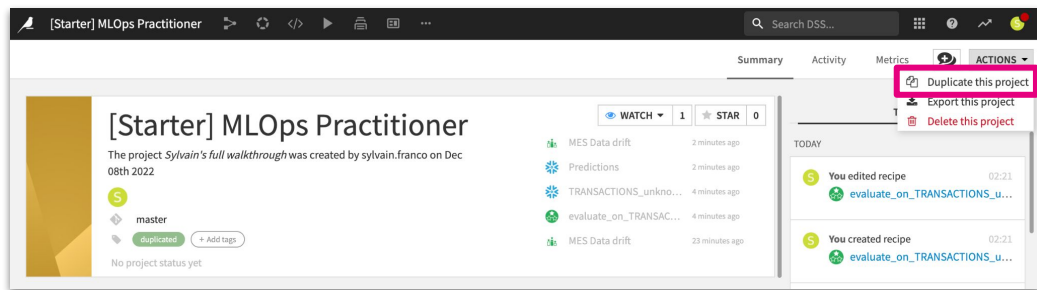


Connect to the Instance and Set up the project

Duplicate the Starter project

TO DO:

- Go to the project **[Starter] {Training session}** > Actions menu > Duplicate this project
- Change the project name and key: **<YOUR NAME> {Training session}**



Duplicate project

Name

Sylvain MLOps Practitioner

Project Key

SYLVAINMLOSPRACTITIONER

The project key is used to reference datasets between projects. It cannot be changed once the project is created.

Project Folder

Dataiku will only copy uploaded datasets and managed folders that are required to rebuild the entire flow. [Advanced options](#)

CANCEL

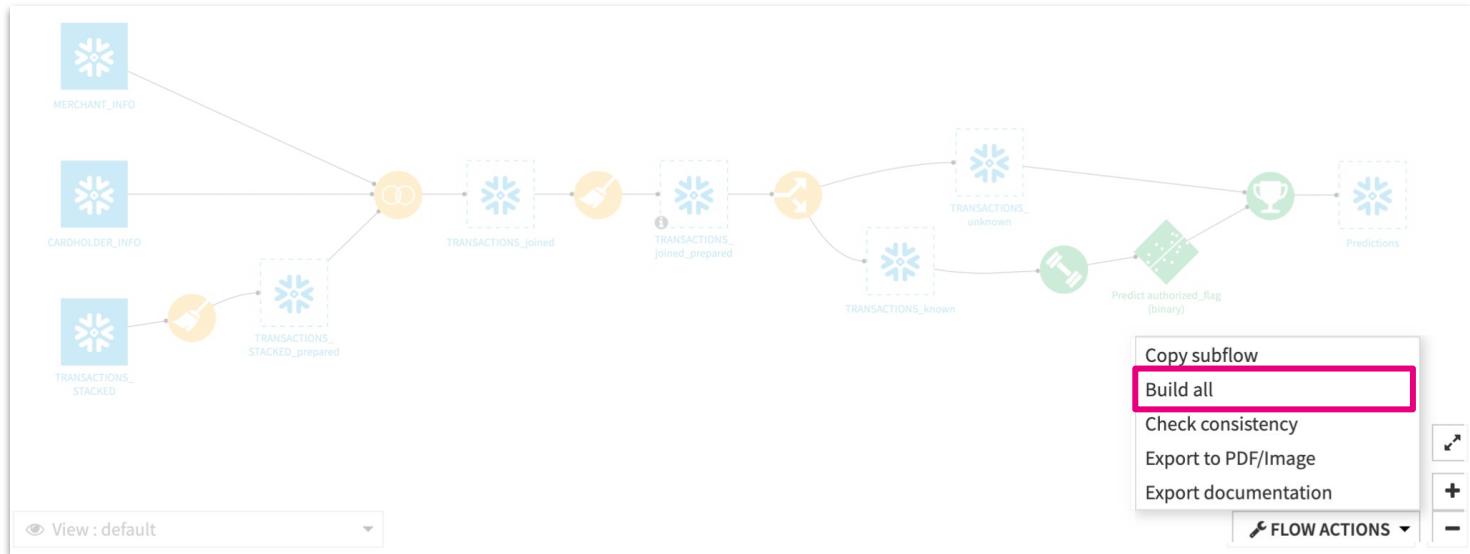
DUPLICATE

Connect to the Instance and Set up the project

Build the flow

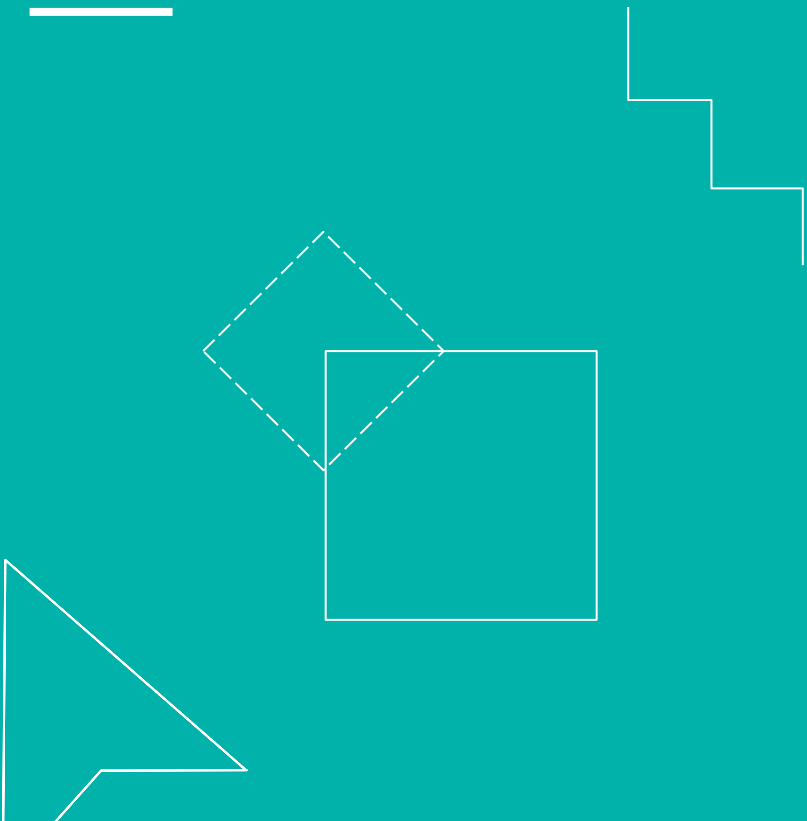
As you can see, most of the datasets in the project* are not built.

Go to Flow Actions -> Build All



* Your starter project may differ from the one shown in this slide

AGENDA

Abstract geometric shapes on a teal background, including a white horizontal line, a white stepped line, a white dashed diamond, a white solid square, and a white triangle.

1. **Background and Objectives**
2. **Modifying the Flow**
 - a. Window recipe
 - b. Schema Propagation
3. **Merchants analysis**
 - a. Group recipe
 - b. TopN recipe
 - c. Pushdown computation
4. **Dataiku Applications**
 - a. Project variables
 - b. Scenarios
 - c. Application Designer

Background and Objectives

Context: Elo Bank



- Major Brazilian debit and credit card brand
- Over 80 million issued cards
- We have data on:
 - Credit card transactions (Jan '21 - Apr '22)
 - Merchant info
 - Credit card holder info

Background and Objectives

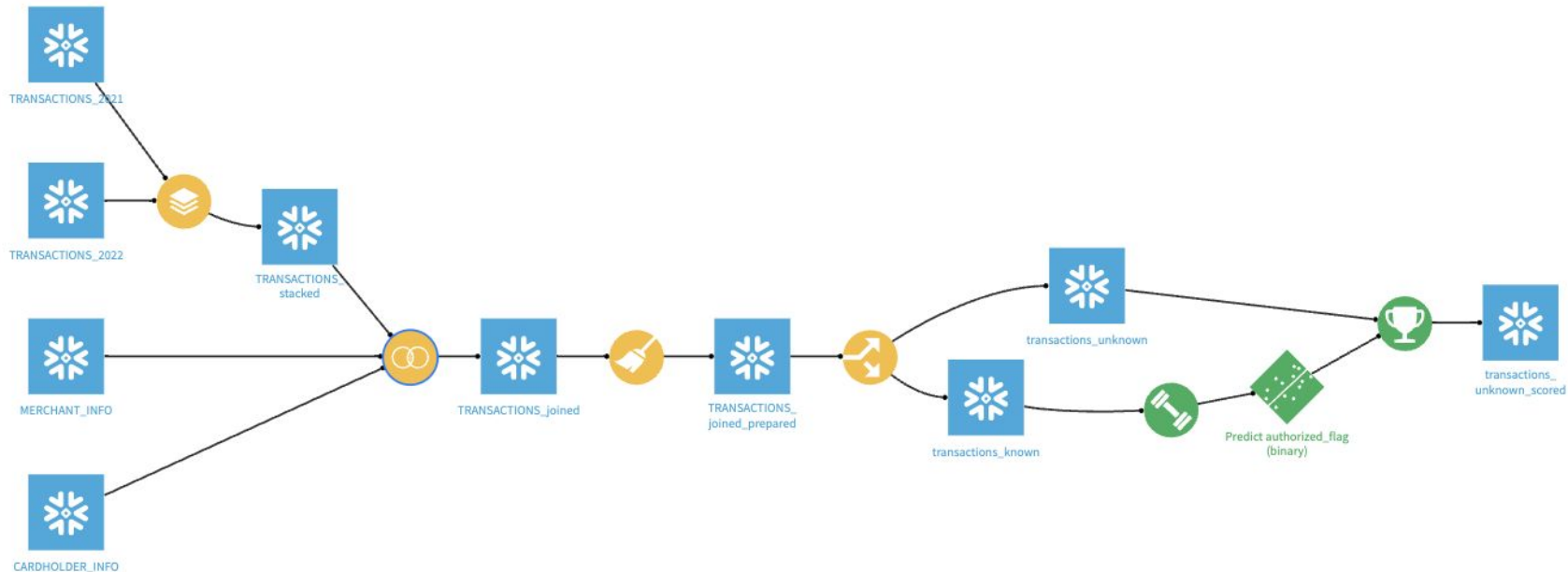
The project so far

Context

- Transactional Data from ELO Bank for the period of January 2021 to April 2022 enriched with client and merchant information
- ML models have been **trained** and reviewed in the lab using **2021** transactional data
- Deployed a Light GBM Model to the flow to **predict** fraud with **2022** transactional data

Background and Objectives

The project so far

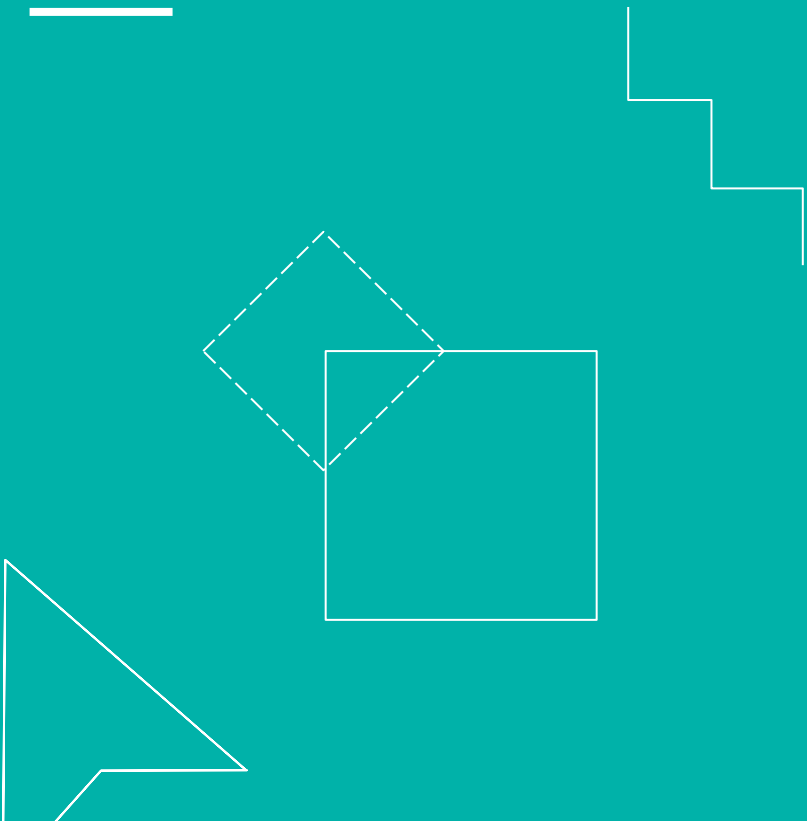


Background and Objectives

Objectives

- Discover and **apply advanced visual recipes** to:
 - Improve ML model accuracy via feature engineering
 - Perform complex analysis
- Improve project **processing performance** by leveraging **pushdown computation**
- Explore **Project Variables** and how they can be applied to a project
- Introduction to the **Automation** capabilities of Dataiku
- Develop a **Dataiku Application**

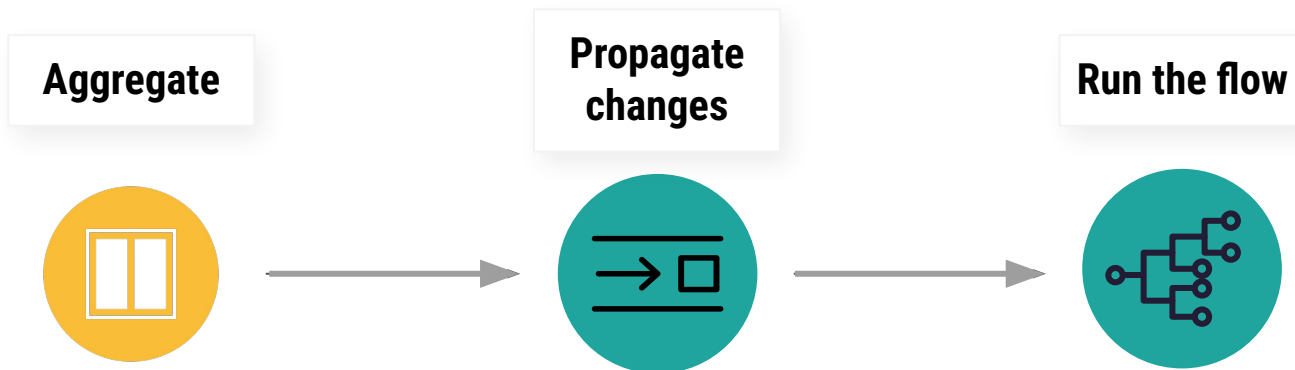
AGENDA

Abstract geometric shapes on a teal background, including a white horizontal line, a white stepped line, a white dashed diamond, a white solid square, and a white triangle.

1. Background and Objectives
2. **Modifying the Flow**
 - a. **Window recipe**
 - b. Schema Propagation
3. Merchants analysis
 - a. Group recipe
 - b. TopN recipe
 - c. Pushdown computation
4. Dataiku Applications
 - a. Project variables
 - b. Scenarios
 - c. Application Designer

Rerouting the Flow

Section Objective: Add New Features



Window



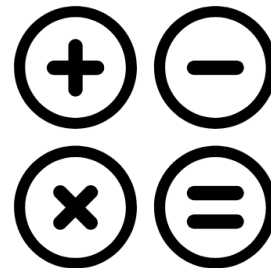
Use this recipe to calculate summary information from groups of rows in a dataset, **while keeping the structure of the dataset intact**

Window recipe

A Window Cousin: The Group By Recipe



1. GROUP KEY



2. AGGREGATIONS

Window recipe

Concept

A **Window** is like a “**flat Group By**”

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	2	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	3	10/14/2017



Grouping key = Customer
Aggregation = Average

CUSTOMER	AVG_AMT
Alice	3.33
Sarah	2
Caroline	3

Window recipe

Concept

A **Window** is like a “**flat Group By**”

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	2	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	3	10/14/2017

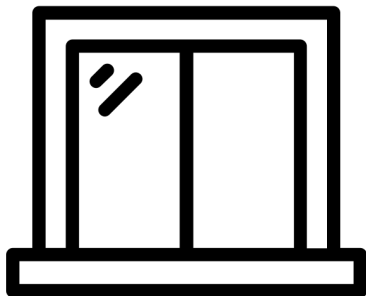


“Grouping key” = Customer
Aggregation = Average

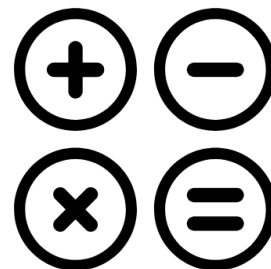
CUSTOMER	AMT	DATE	AVG_AMT
Alice	5	10/1/2017	3.33
Sarah	1	10/1/2017	2
Sarah	4	10/2/2017	2
Alice	2	10/5/2017	3.33
Caroline	4	10/7/2017	3
Caroline	2	10/13/2017	3
Sarah	1	10/13/2017	2
Alice	3	10/14/2017	3.33

Window recipe

Components



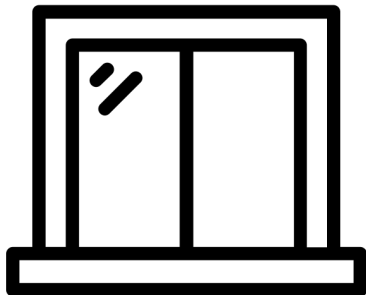
1. WINDOW DEFINITION



2. AGGREGATIONS

Window recipe

Components



1. WINDOW DEFINITION

Window definition:

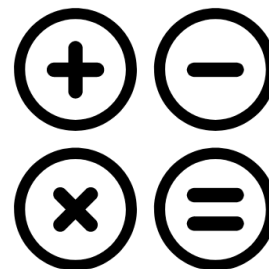
- **Partitioning column** (or column to group by)
- **Ordering column** (Optional)
- **Window frame** based on the ordering column (Optional)

Window recipe

Components

More complex aggregations:

- **Rank**
- **Lags**
- **Rolling averages**
- ...



2. AGGREGATIONS

Window recipe

Aggregations: Rank

Rank: is it the first, second, third, ... order of the customer ?

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	2	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	2	10/14/2017



Window partition = Customer
Ordering column = Date

CUSTOMER	AMT	DATE	RANK
Alice	5	10/1/2017	1
Alice	4	10/5/2017	2
Alice	2	10/14/2017	3
Sarah	1	10/1/2017	1
Sarah	4	10/2/2017	2
Sarah	1	10/13/2017	3
Caroline	4	10/7/2017	1
Caroline	2	10/13/2017	2

Window recipe

Aggregations: Lag features

Lag: what was the amount of the previous order ?

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	2	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	2	10/14/2017



Window partition = Customer
Ordering column = Date

CUSTOMER	AMT	DATE	LAG AMT
Alice	5	10/1/2017	-
Alice	4	10/5/2017	5
Alice	2	10/14/2017	4
Sarah	1	10/1/2017	-
Sarah	4	10/2/2017	1
Sarah	1	10/13/2017	4
Caroline	4	10/7/2017	-
Caroline	2	10/13/2017	4

Window recipe

Aggregations: Rolling Average

Rolling Average: customer's average order quantity over the previous three days

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	2	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	2	10/14/2017



Window partition = Customer
Ordering column = Date

CUSTOMER	AMT	DATE	ROLL AVG
Alice	5	10/1/2017	5
Alice	4	10/5/2017	4.5
Alice	2	10/14/2017	3.6667
Sarah	1	10/1/2017	1
Sarah	4	10/2/2017	2.5
Sarah	1	10/13/2017	2
Caroline	4	10/7/2017	4
Caroline	2	10/13/2017	3

Window recipe

Aggregations: Rolling Average

Rolling Average: customer's average order quantity over the previous three days

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	2	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	2	10/14/2017



Window partition = Customer
Ordering column = Date

CUSTOMER	AMT	DATE	ROLL AVG
Alice	5	10/1/2017	5
Alice	4	10/5/2017	4.5
Alice	2	10/14/2017	3.6667
Sarah	1	10/1/2017	1
Sarah	4	10/2/2017	2.5
Sarah	1	10/13/2017	2
Caroline	4	10/7/2017	4
Caroline	2	10/13/2017	3

Window recipe

Aggregations: Rolling Average

Rolling Average: customer's average order quantity over the previous three days

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	2	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	2	10/14/2017



Window partition = Customer
Ordering column = Date

CUSTOMER	AMT	DATE	ROLL AVG
Alice	5	10/1/2017	5
Alice	4	10/5/2017	4.5
Alice	2	10/14/2017	3.6667
Sarah	1	10/1/2017	1
Sarah	4	10/2/2017	2.5
Sarah	1	10/13/2017	2
Caroline	4	10/7/2017	4
Caroline	2	10/13/2017	3

Window recipe

Aggregations: Rolling Average

Rolling Average: customer's average order quantity over the previous three days

CUSTOMER	AMT	DATE
Alice	5	10/1/2017
Sarah	1	10/1/2017
Sarah	4	10/2/2017
Alice	4	10/5/2017
Caroline	4	10/7/2017
Caroline	2	10/13/2017
Sarah	1	10/13/2017
Alice	2	10/14/2017



Window partition = Customer
Ordering column = Date

CUSTOMER	AMT	DATE	ROLL AVG
Alice	5	10/1/2017	5
Alice	4	10/5/2017	4.5
Alice	2	10/14/2017	3.6667
Sarah	1	10/1/2017	1
Sarah	4	10/2/2017	2.5
Sarah	1	10/13/2017	2
Caroline	4	10/7/2017	4
Caroline	2	10/13/2017	3

Window recipe

Compute new features

Objective: create features based on each client's and each merchant's transaction history.

For **each client**, for **each transaction** :

- **Average** amount of all transactions **up to** the given transaction
- **Minimum** amount of all transactions **up to** the given transaction
- **Maximum** amount of all transactions **up to** the given transaction

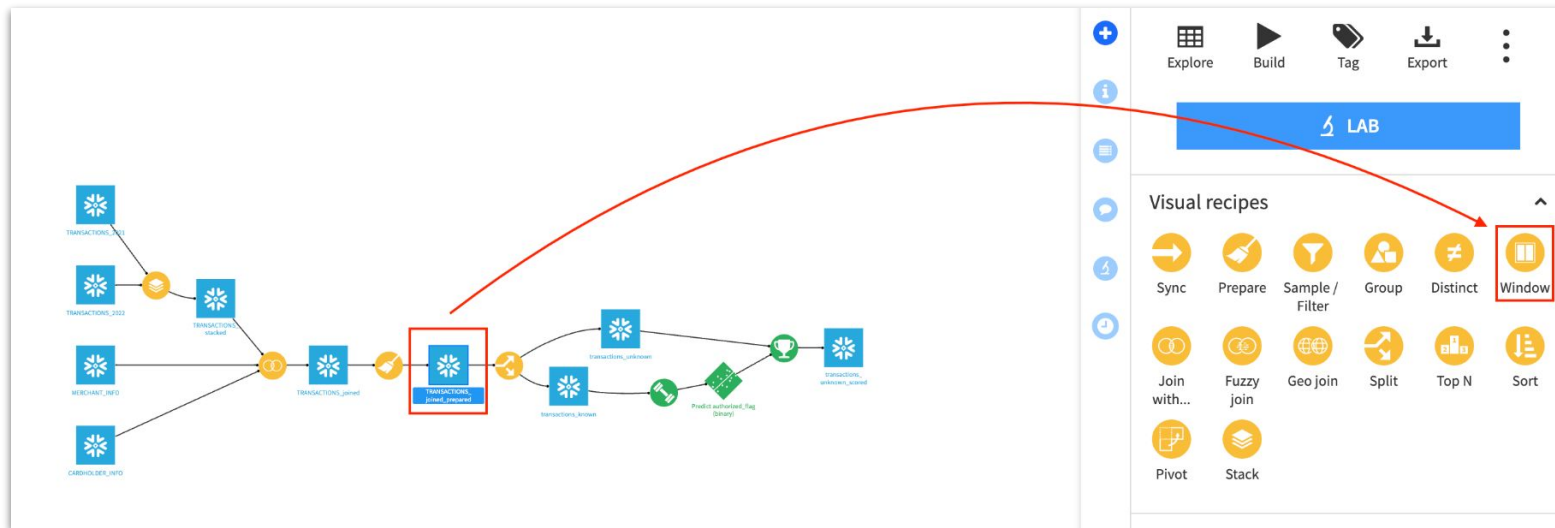
For **each merchant**, for **each transaction** :

- **Average** amount of all transactions **up to** the given transaction
- **Minimum** amount of all transactions **up to** the given transaction
- **Maximum** amount of all transactions **up to** the given transaction

Window recipe

Create the recipe

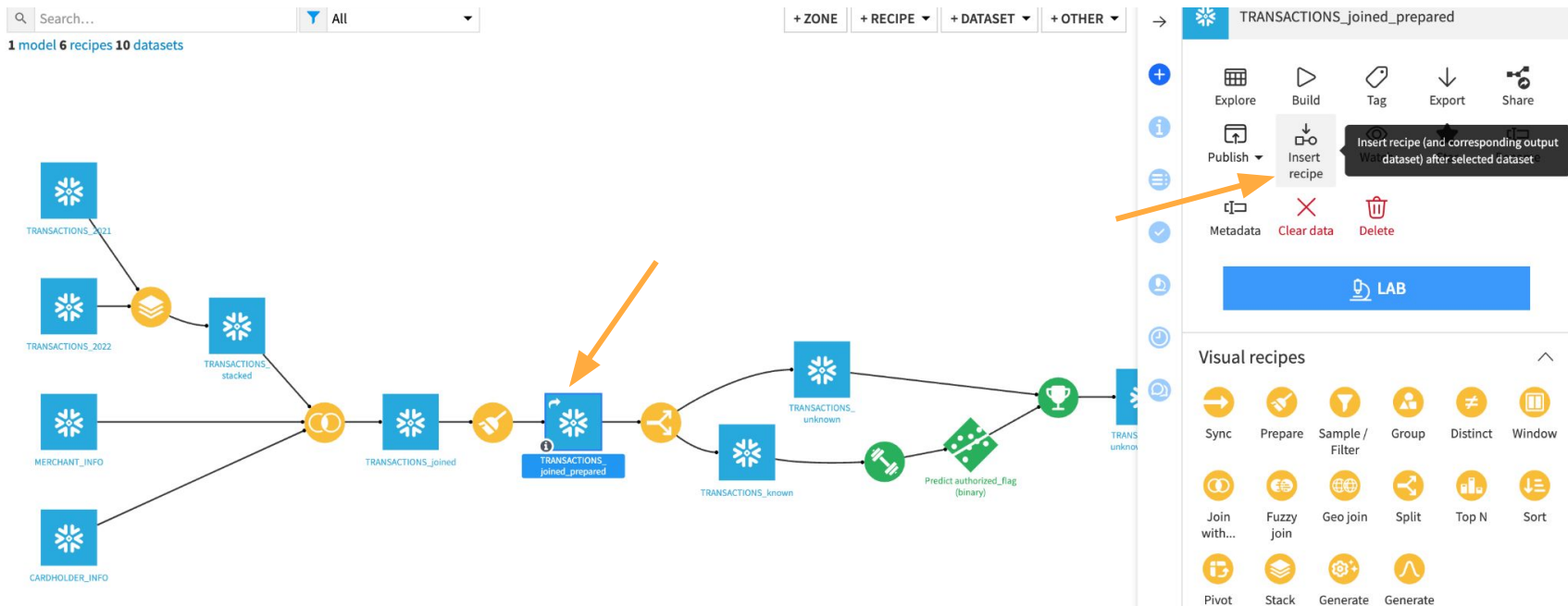
TO DO: create a Window Recipe on the *transactions_joined_prepared* dataset, but we want to keep the Flow order



Option 1: Insert the Window recipe

Insert recipe

TO DO: create a Window Recipe on the *transactions_joined_prepared* dataset.



Option 1: Insert the Window recipe

Insert recipe

The screenshot shows the 'Insert recipe after' dialog in Dataiku. The title bar reads 'Insert recipe after "TRANSACTIONS_joined_pr...'. The main area displays a recipe graph with a blue square node, a yellow circle node, and a light blue square node. A context menu is open over the yellow circle node, listing recipe types: Visual, Code, Hadoop & Spark, Data preparation (selected), Sync, Sample / Filter, Distinct, Window, Join, Fuzzy join, Geo join, Split, Top N, and Sort. Below the menu, the 'Recipe type' dropdown is set to 'Select recipe type'. A message states: 'The selected dataset has a unique subsequent recipe. The output dataset of this recipe will be configured as an input of the following r'. Below this, a yellow icon and the text 'split_TRANSACTIONS_joined_prepared' are visible. At the bottom are 'CANCEL' and 'NEXT' buttons.

The screenshot shows the 'New window recipe' configuration dialog. It is divided into two main sections: 'Input dataset' and 'Output dataset'. In the 'Input dataset' section, a dropdown menu shows 'TRANSACTIONS_joined_prepared' with a link to 'VIEW'. In the 'Output dataset' section, the 'Name' field contains 'TRANSACTIONS_joined_prepared_windows'. The 'Store into' dropdown is set to 'snowflake'. Below these fields, a 'NEW DATASET' section contains an information icon and a message: 'This dataset will be reconfigured as an input to the original downstream recipe.' At the bottom are 'CANCEL' and 'CREATE RECIPE' buttons.

Option 1: Insert the Window recipe

Define the Windows

TO DO: define two windows :

- one for cardholders
- the other for merchants.

Each window should:

- Have an appropriate prefix
- be ordered by *purchase_date_parsed*
- be framed to include previous rows only.

The screenshot displays the 'Window description' configuration for two windows in a Dataiku recipe. Both windows are configured with the following settings:

- Window description:** The prefix is highlighted in a red box. For the top window, it is 'card'; for the bottom window, it is 'merchant'.
- PARTITIONING COLUMNS:** The 'ON' toggle is active. The top window has 'card_id' selected; the bottom window has 'merchant_id' selected. Both have a '+ ADD COLUMN' button.
- ORDER COLUMNS:** The 'ON' toggle is active. Both windows have 'purchase_date_parsed' selected. Both have a '+ ADD COLUMN' button.
- WINDOW FRAME:** The 'ON' toggle is active. A blue information box states: 'Window frames enable you to limit the number of rows taken into account to compute aggregations. Example: compute sliding averages for time series.' Below this, a dropdown menu is set to 'Limit the number of preceding/following rows'. The 'Limit following rows' checkbox is checked, and the 'Number of rows after' is set to 0.

Option 1: Insert the Window recipe

Define the Window Aggregations

TO DO: in the *Aggregations* panel of the Window Recipe, enable the *min*, *max* and *avg* aggregations for the *purchase_amount* column; *run* the recipe.

Pre-filter

Computed columns

Windows definitions

Aggregations

Custom aggregations

▶ RUN
⚙

In-database (SQL)

Compute rank for each row

You can compute the "ranks" of each row, according the ordering defined in the [windows definitions](#), with the following methods.

A rank is computed for each defined window.

Compute:
 Rank
Dense rank
Row number
Cumulative distribution
Quantile

Compute aggregations for each field

Each aggregation will be computed separately within each window.

Ex: if you defined 2 windows, selecting "min" for a column will compute two different aggregations.

☐ 0 / 33

Filter column...

☐ Hide unused variables

☐ item_category	string	Retrieve	Min	Max	Avg	Sum	Std-dev:	Count	First	Last	Concat	options ▼	Lag	...
☐ purchase_amount	double	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼	Lag	...
☐ signature_provided	bigint	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼	Lag	...
☐ merchant_subsector_desc...	string	Retrieve	Min	Max	Avg	Sum	Std-dev:	Count	First	Last	Concat	options ▼	Lag	...
☐ spending_type	string	Retrieve	Min	Max	Avg	Sum	Std-dev:	Count	First	Last	Concat	options ▼	Lag	...
☐ merchant_latitude	double	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼	Lag	...
☐ merchant_longitude	double	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼	Lag	...
☐ card_first_active_month	string	Retrieve	Min	Max	Avg	Sum	Std-dev:	Count	First	Last	Concat	options ▼	Lag	...

Option 1: Insert the Window recipe

Save and Update the Output Schema

TO DO: save the changes, update the schema of the output datasets, and run downstream.

The screenshot displays the Dataiku interface with a recipe execution dialog open. The background shows a search bar, a filter dropdown set to 'All', and a summary of '1 model 7 recipes 11 datasets'. Below this, a flow diagram is partially visible with recipes named 'TRANSACTIONS_joined', 'TRANSACTIONS_joined_prepared', and 'TRANSACTIONS_joined_prepared_wi'. The dialog box is titled 'Run recipe "compute_TRANSACTIONS_joined_prepared_wi..."'. It features two buttons: 'RUN ONLY THIS' and 'RUN DOWNSTREAM'. Below the buttons is a visual representation of the recipe flow, showing a sequence of steps with a blue location pin icon on the third step. The dialog includes instructions: 'From this recipe, run all recipes and build all datasets from left to right until the end of the Flow.' It also has settings for 'Update output schemas' (checked) and 'Mode' (set to 'Build all downstream'). A 'RUN' button is at the bottom right of the dialog. On the right side of the interface, a sidebar contains icons for 'Edit', 'Run', 'Tag', 'Change engine', 'Copy', 'Unwatch', 'Star', 'Metadata', and 'Delete', followed by sections for 'Flow Zones' and 'Other actions'.

Search... All
1 model 7 recipes 11 datasets

TRANSACTIONS_joined
TRANSACTIONS_joined_prepared
TRANSACTIONS_joined_prepared_wi

Run recipe "compute_TRANSACTIONS_joined_prepared_wi..."

RUN ONLY THIS RUN DOWNSTREAM

From this recipe, run all recipes and build all datasets from left to right until the end of the Flow.

Update output schemas ☒ Before the recipe runs, update the schema of the output dataset(s)

Mode ☒ Build all downstream ☐ Find outputs and build recursively

RUN

compute_TRANSACTIONS_joined_prepared_wi windows

Edit Run Tag Change engine Copy
Unwatch Star Metadata Delete

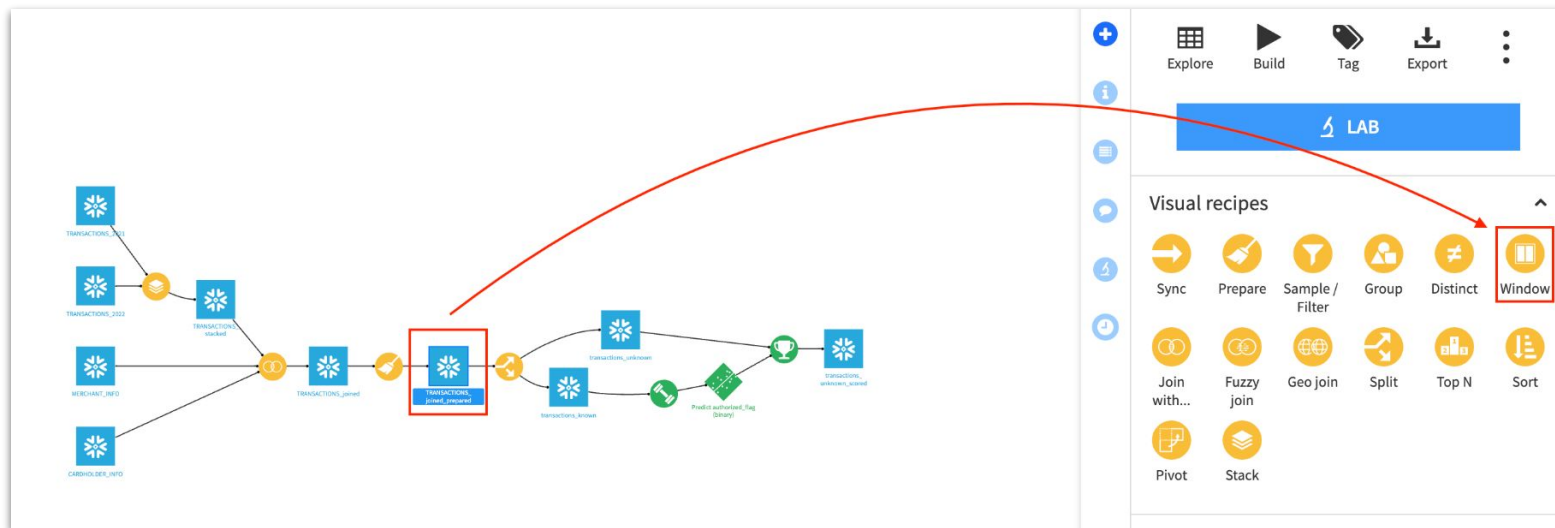
Flow Zones
Move

Other actions
Change Spark configuration
Change Spark engine
Change Spark pipelineability
Change SQL pipelineability

Create the Window recipe

Create the recipe

TO DO: create a Window Recipe on the *transactions_joined_prepared* dataset.



Create the Window recipe

Define the Windows

TO DO: define two windows :

- one for cardholders
- the other for merchants.

Each window should:

- Have an appropriate prefix
- be ordered by *purchase_date_parsed*
- be framed to include previous rows only.

The screenshot displays the 'Define the Windows' configuration interface, which is divided into two sections for defining different windows. Each section includes a 'Window description' header, a 'Prefix' field, and three main configuration areas: 'PARTITIONING COLUMNS', 'ORDER COLUMNS', and 'WINDOW FRAME'.

Window 1: card

- Window description:** Prefix: card
- PARTITIONING COLUMNS:** ON. Includes a dropdown menu with 'card_id' and a '+ ADD COLUMN' button.
- ORDER COLUMNS:** ON. Includes a dropdown menu with 'purchase_date_parsed' and a '+ ADD COLUMN' button.
- WINDOW FRAME:** ON. Includes a dropdown menu for 'Limit the number of preceding/following rows' and checkboxes for 'Limit preceding rows' (unchecked) and 'Limit following rows' (checked). The 'Number of rows after' field is set to 0.

Window 2: merchant

- Window description:** Prefix: merchant
- PARTITIONING COLUMNS:** ON. Includes a dropdown menu with 'merchant_id' and a '+ ADD COLUMN' button.
- ORDER COLUMNS:** ON. Includes a dropdown menu with 'purchase_date_parsed' and a '+ ADD COLUMN' button.
- WINDOW FRAME:** ON. Includes a dropdown menu for 'Limit the number of preceding/following rows' and checkboxes for 'Limit preceding rows' (unchecked) and 'Limit following rows' (checked). The 'Number of rows after' field is set to 0.

Create the Window recipe

Define the Window Aggregations

TO DO: in the *Aggregations* panel of the Window Recipe, enable the *min*, *max* and *avg* aggregations for the *purchase_amount* column; *run* the recipe.

The screenshot shows the Dataiku Window Recipe interface. On the left, a vertical pipeline diagram includes steps: Pre-filter, Computed columns, Windows definitions (marked with a green check), Aggregations (marked with a green check), and Custom aggregations. At the bottom of the pipeline are 'RUN' and 'In-database (SQL)' buttons.

The main panel is titled 'Compute rank for each row' and contains a sub-section 'Compute aggregations for each field'. Below this, there is a search bar and a table of aggregation options for various columns.

Compute aggregations for each field

Each aggregation will be computed separately within each window.
Ex: if you defined 2 windows, selecting "min" for a column will compute two different aggregations.

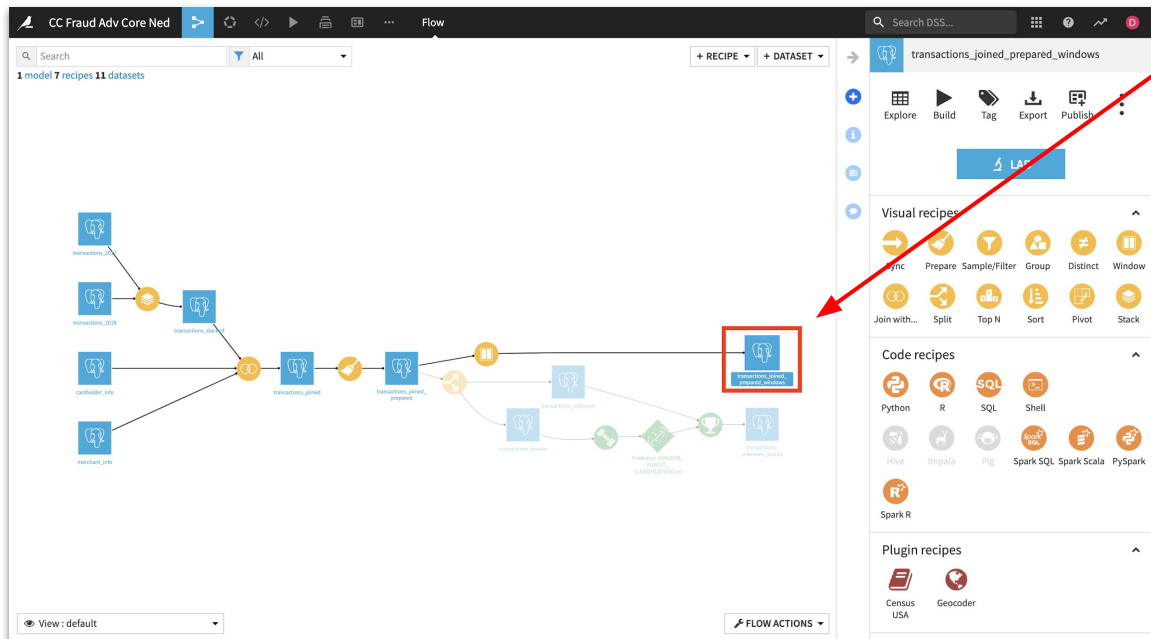
0 / 33 Filter column... Hide unused variables

Column	Type	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
item_category	string	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
purchase_amount	double	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
signature_provided	bigint	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
merchant_subsector_desc...	string	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
spending_type	string	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
merchant_latitude	double	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
merchant_longitude	double	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La
card_first_active_month	string	Retrieve	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options	Lag	La

Create the Window recipe

Inspect the Newly Added Features

TO DO: inspect the *transactions_joined_prepared_windows* dataset, to ensure the new features have been successfully added.

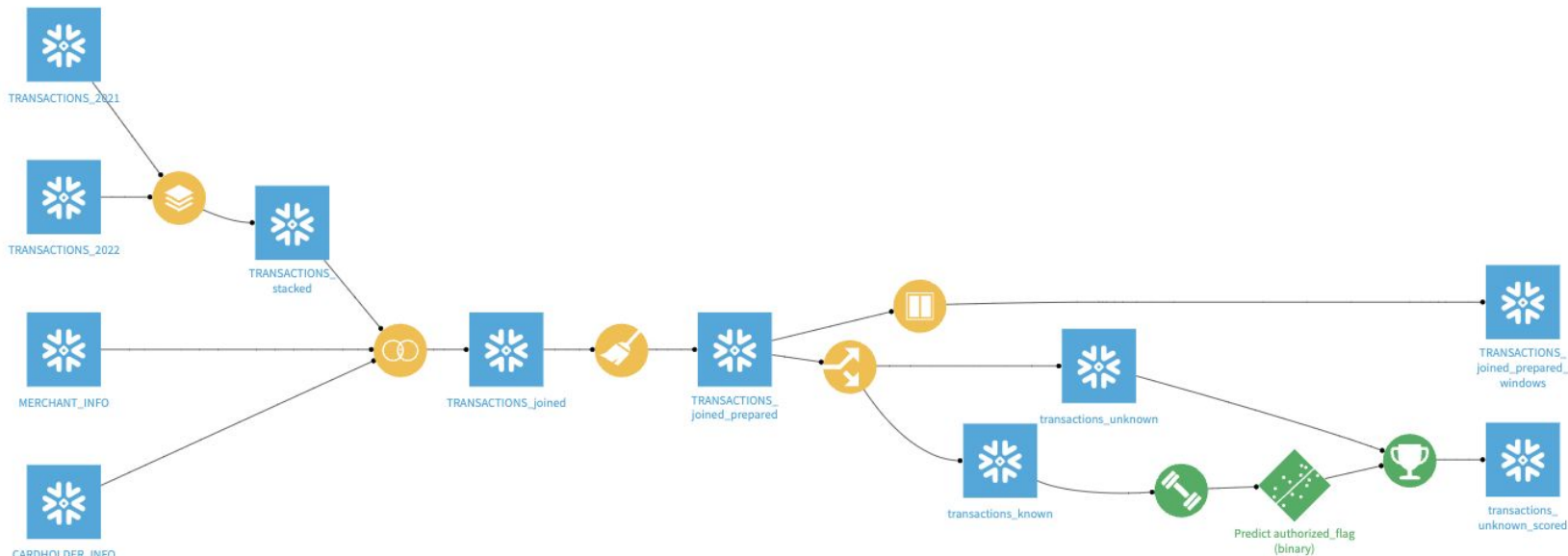


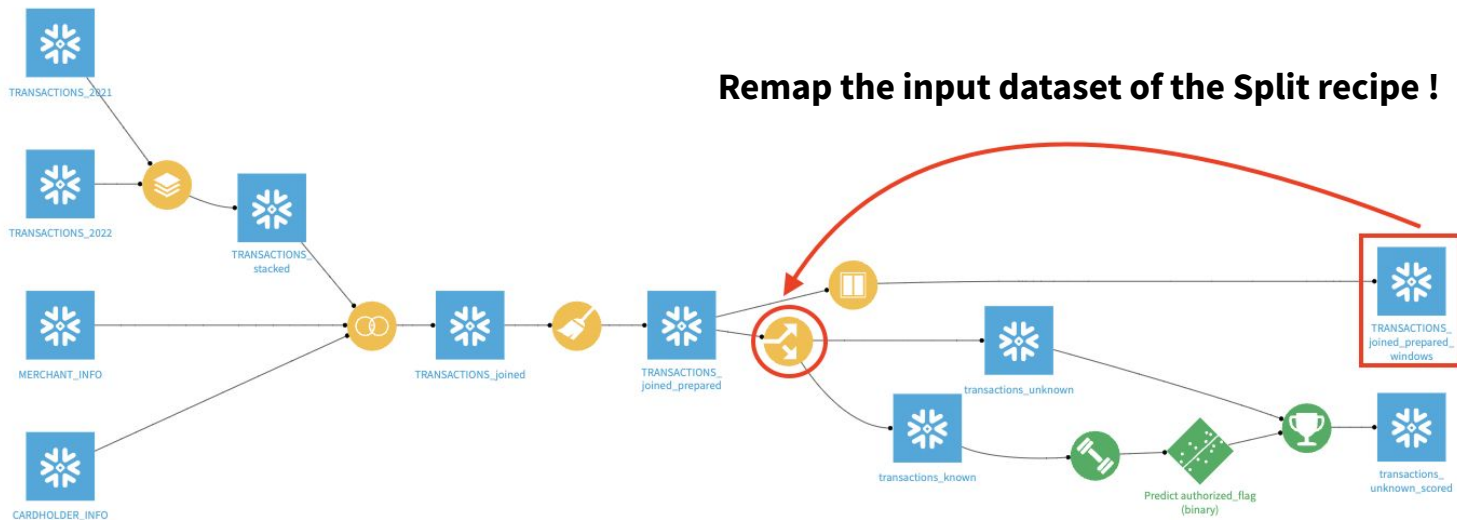
	card_purchase_amount_avg	merchant_purchase_amount_min
	double Decimal	double Decimal
21	423.0728571428571	138.63
38	309.46666666666664	787.38
53	584.53	584.53
05	254.43874999999997	584.53
04	126.34714285714287	26.37
04	117.12500000000001	26.37
91	186.10333333333335	26.37
35	18.35	18.35
35	18.35	18.35
35	18.35	18.35
0.4	990.4	18.35
0.4	615.03	18.35
37	223.73000000000002	302.76
0.4	541.6066666666667	50.03
82	14.82	14.82
76	14.58	10.65

Remap Input / Output

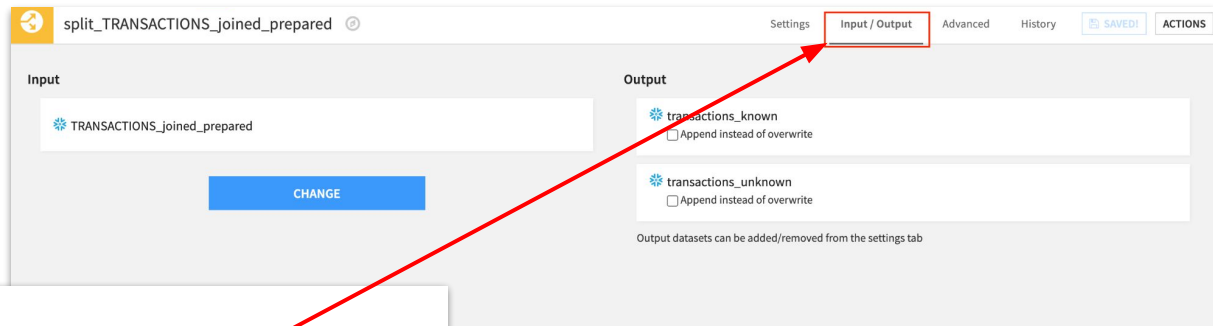
Include the New Features in ETL Pipeline?

Now that we've created a dataset with new features, we should include them in the ETL pipeline leading up to the machine learning model... but how can we do this?





TO DO: navigate to the Split Recipe *Input / Output* tab.



Create the Window recipe

Remap the Split Recipe Input Dataset

TO DO: remap the Split Recipe input dataset, by clicking on the blue *Change* button under the *transactions_joined_prepared* dataset. Then select *transactions_joined_prepared_windows* from the dropdown.

The screenshot shows the configuration interface for a Dataiku recipe named 'split_TRANSACTIONS_joined_prepared'. The interface is divided into two main sections: 'Input' and 'Output'.

Input Section:

- A search bar is at the top.
- A list of input datasets is shown below. The dataset 'TRANSACTIONS_joined_prepared_windows' is highlighted with a red box, indicating it is the selected input dataset.
- Other input datasets listed include: MERCHANT_INFO, TRANSACTIONS_2021, TRANSACTIONS_2022, TRANSACTIONS_joined, transactions_known, TRANSACTIONS_stacked, transactions_unknown, transactions_unknown_scored, and Predict authorized_flag (binary).
- A note at the bottom states: 'This input doesn't accept saved models'.

Output Section:

- Two output datasets are listed: 'transactions_known' and 'transactions_unknown'.
- Each output dataset has a checkbox labeled 'Append instead of overwrite'.
- A note below the output datasets states: 'Output datasets can be added/removed from the settings tab'.

The top navigation bar includes tabs for 'Settings', 'Input / Output', 'Advanced', and 'History'. There are also buttons for 'SAVED!' and 'ACTIONS'.

Create the Window recipe

Save and Update the Output Schema

TO DO: save the changes, and update the schema of the output datasets.

Schema changes

The current schema of output dataset(s) doesn't match what the recipe outputs.

transactions_known

Different number of columns (33 columns previously, 39 columns in the current recipe)

It is recommended to drop the data of this dataset to avoid schema inconsistencies.

☒ Drop and recreate

transactions_unknown

Different number of columns (33 columns previously, 39 columns in the current recipe)

It is recommended to drop the data of this dataset to avoid schema inconsistencies.

☒ Drop and recreate

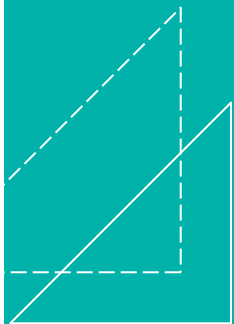
CANCEL SAVE

IGNORE

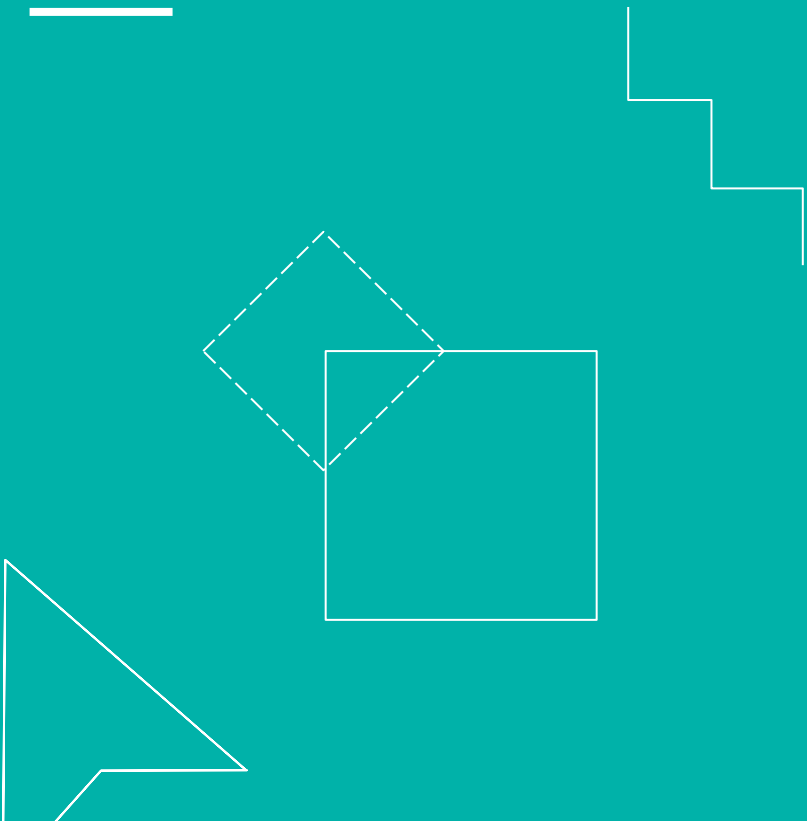
UPDATE SCHEMA



**We have rerouted our Flow.
Did we break anything in the
process ?**



AGENDA

Abstract geometric shapes on a teal background, including a white horizontal line, a white stepped line, a white dashed diamond, a white solid square, and a white triangle.

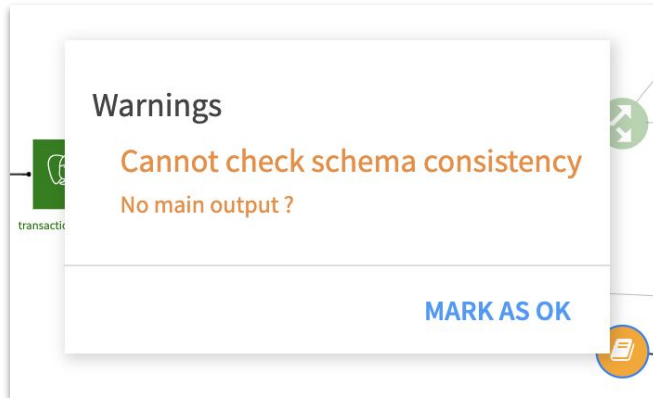
1. Background and Objectives
2. Modifying the Flow
 - a. Window recipe
 - b. Schema Propagation**
3. Merchants analysis
 - a. Group recipe
 - b. TopN recipe
 - c. Pushdown computation
4. Dataiku Applications
 - a. Project variables
 - b. Scenarios
 - c. Application Designer

Schema Propagation

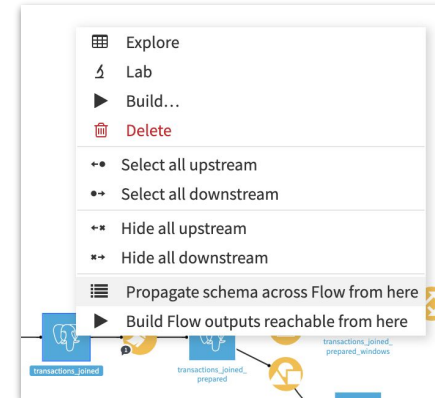
Tools available to work with schemas

DSS provides two utilities for working with schemas:

1) Check Consistency



2) Propagate schema across flow from here



Schema Propagation

Tools available to work with schemas

Consistency Checks allow users to have :

- Verification of schema consistency within the project flow.
- Alerts should there be any inconsistencies downstream

To help with updating your flow, DSS provides a **Schema Propagation tool** that will allow users to automatically update all schemas downstream from a selected point in the flow.

Schema Propagation

Check Project Consistency

TO DO: navigate to the project flow, and select *FLOW ACTIONS* → *Check Consistency*, and then click “Start Check” to execute the consistency check.



Consistency check

Options

START CHECK

Perform potentially slow checks in recipes

Check consistency between dataset definition and data

Check consistency between recipe and dataset schema

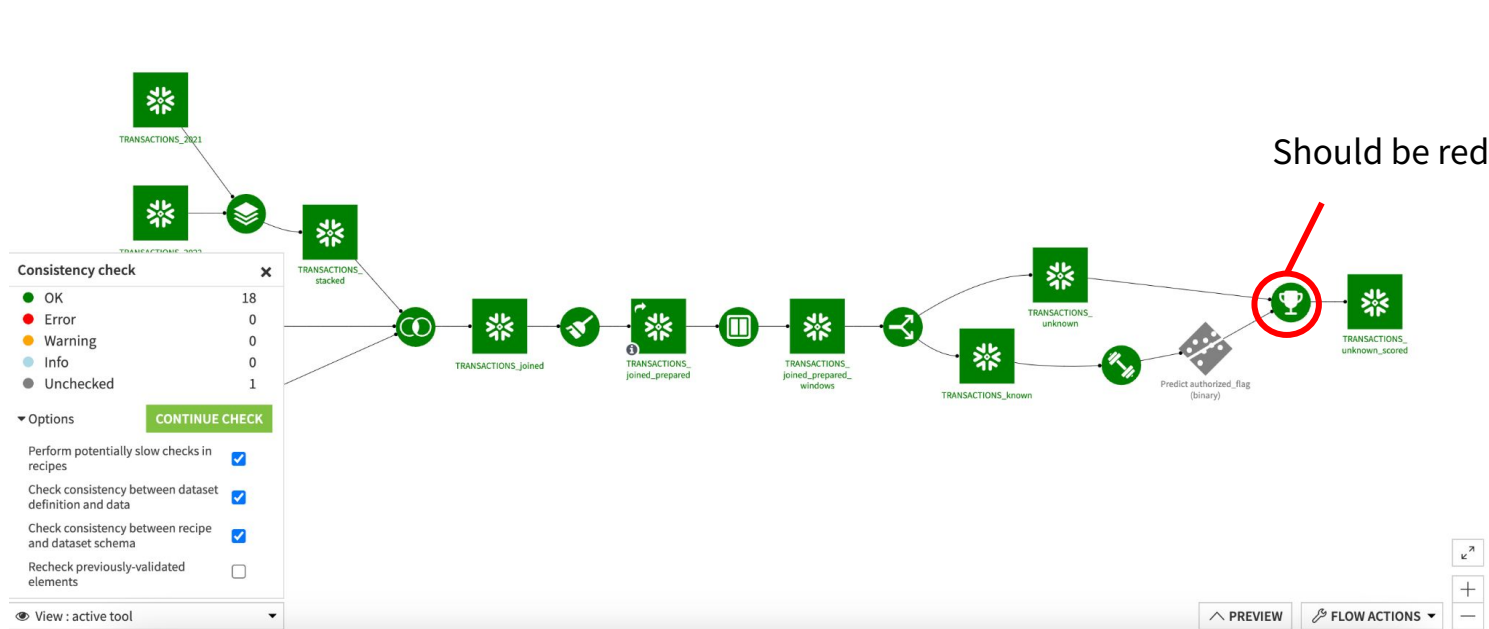
Recheck previously-validated elements

View : active tool

Schema Propagation

Check Project Consistency

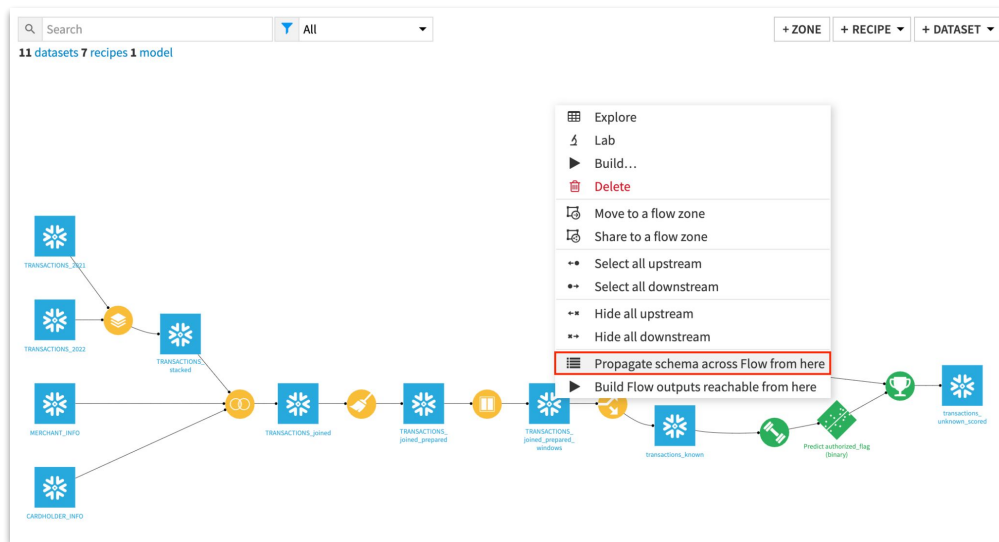
TO DO: See that the Consistency check passed.



Optional: Schema Propagation

Propagate the schema

TO DO: right-click on the *transactions_joined_prepared_windows* dataset, and select “Propagate schema across flow from here”, and then click “Start” to run the tool.



Schema propagation ✕

● OK

0

● Needs update

0

● Needs manual check

0

● Not yet checked

3

● Excluded from propagation

0

START

Perform all actions automatically

☐

Perform all actions automatically and build all outputs datasets afterwards

☐

More options

View : active tool

▼

Schema Propagation

View Column usage

TO DO: Check which datasets contain a particular newly added column, eg `card_purchase_amount_min` and whether that column has been propagated in downstream datasets

Search... All

1 model 7 recipes 11 datasets

+ ZONE + RECIPE + DATASET + OTHER

TRANSACTIONS_joined_prepared_windows

merchant_latitude double
merchant_longitude double
card_first_active_month string
card_first_active_month_parsed date
days_active bigint
card_reward_program string
card_latitude double
card_longitude double
card_fico_score bigint
card_age bigint
merchant_location string
merchant_state string
merchant_state_enName string
card_location string
distance_card_merchant double
card_state string
card_state_enName string
card_purchase_amount_min double
card_purchase_amount_max double
card_purchase_amount_avg double
merchant_purchase_amount_min double
merchant_purchase_amount_max double
merchant_purchase_amount_avg double

Column usage

☒ card_purchase_amount_min 4

Add column... ADD

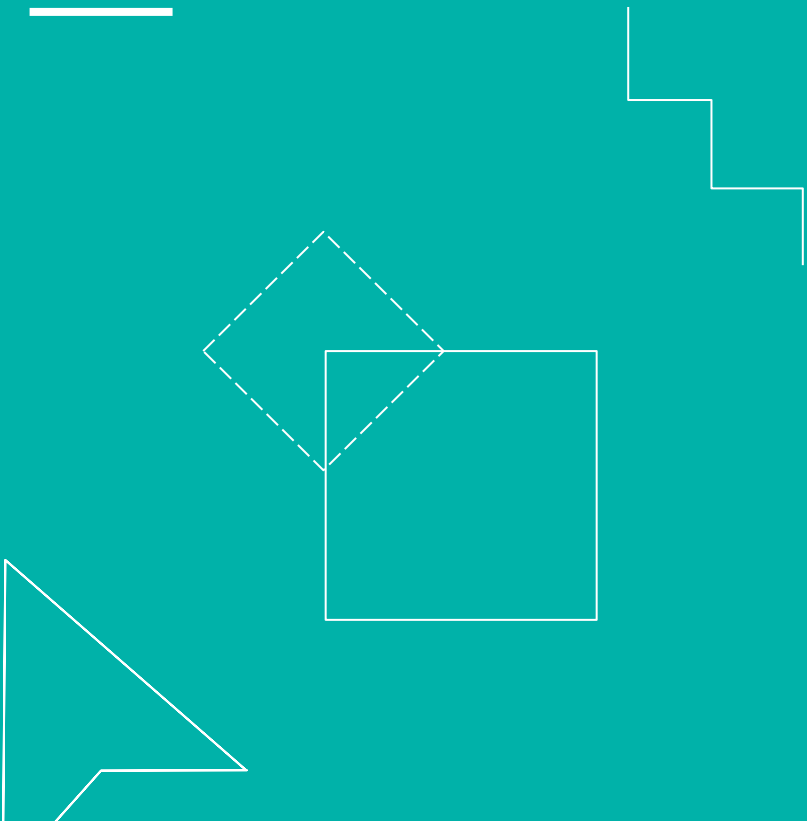
Case sensitive ☐

View : Column usage

PREVIEW FLOW ACTIONS

The screenshot displays the Dataiku interface for schema propagation. At the top, there's a search bar and filters for ZONE, RECIPE, DATASET, and OTHER. Below this is a data lineage graph showing the flow from various input datasets (TRANSACTIONS, TRANSACTIONS_RAW, MERCHANT_RAW, CARDHOLDER_RAW) through several intermediate datasets (TRANSACTIONS_joined, TRANSACTIONS_joined_prepared, TRANSACTIONS_joined_prepared_windows, TRANSACTIONS_joined_prepared_windows_parsed, TRANSACTIONS_joined_prepared_windows_parsed_parsed) to a final output dataset (TRANSACTIONS_joined_prepared_windows_parsed_parsed_parsed). On the right, a table lists the columns and their data types for the 'TRANSACTIONS_joined_prepared_windows' dataset. At the bottom left, a 'Column usage' panel shows a list of columns, with 'card_purchase_amount_min' selected and highlighted by a red arrow. The panel also includes an 'Add column...' button, a 'Case sensitive' checkbox, and a 'View : Column usage' dropdown.

AGENDA



1. Background and Objectives
2. Modifying the Flow
 - a. Window recipe
 - b. Schema Propagation
3. **Merchants analysis**
 - a. **Group recipe**
 - b. **TopN recipe**
 - c. Pushdown computation
4. Dataiku Applications
 - a. Project variables
 - b. Scenarios
 - c. Application Designer

Merchant Analysis

Merchants with a high fraud percentage : Group by, Join and topN

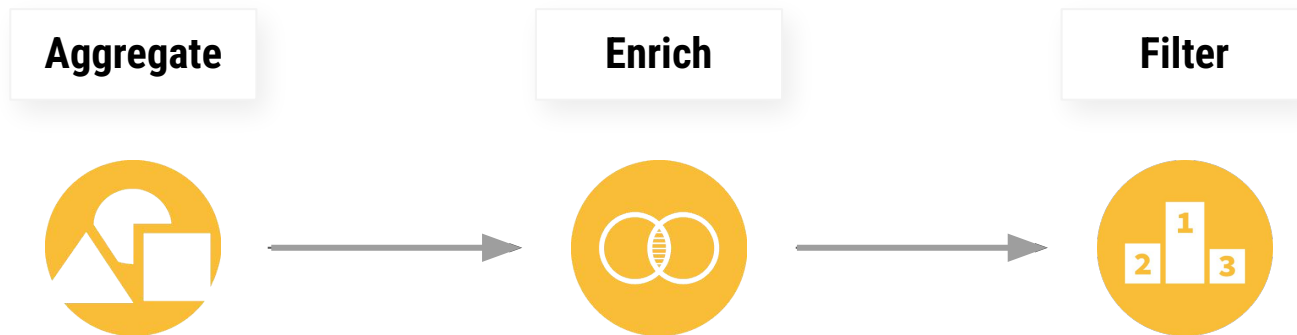
Let's identify the merchants which have registered the **highest amount of fraud** (as a percentage of their overall revenue).



merchant_id	merchant_name	merchant_turnover	Sum_fraudulent_transactions	percent_fraud
string	string	double	double	double
Text	Text	Decimal	Decimal	Decimal
M_ID_01f4e277b0	Wootton Retail	87102.0	48390.0	0.5555555555555556
M_ID_9f227ee53d	Sunset Imports	98871.444	41781.28	0.4225818730835973
M_ID_5db0c74036	New Look Wholesale	172004.022	51682.51999999999	0.3004727412711314
M_ID_80e51390c6	Kerby's Retailer	220467.474	61052.4	0.27692248154482874
M_ID_3111c6df35	Biltmore Menswear	1177293.6179999996	240143.76	0.2039794969822049
M_ID_c9855bf3e3	Druxy Boots	337256.24399999983	65086.34	0.1929877983222752
M_ID_b9f9332438	Fairbank Direct	291553.47	54451.16	0.1867621743620476
M_ID_9e84cda3b1	Indicohosts	226692.2880000002	41945.92	0.18503461396975251
M_ID_445742726b	Blvd Media	952645.3200000001	157761.69	0.16560380520212914
M_ID_9dbbaabb0a	Payless ShoeSource	140378.54400000014	23088.10000000002	0.16447029112939068

Merchant Analysis

Objective



Merchant Analysis

Group by

Let's start by aggregating the **sum of fraudulent sales by merchant**



merchant_id	merchant_name	merchant_turnover	Sum_fraudulent_transactions	percent_fraud
string	string	double	double	double
Text	Text	Decimal	Decimal	Decimal
M_ID_01f4e277b0	Wootton Retail	87102.0	48390.0	0.5555555555555556
M_ID_9f227ee53d	Sunset Imports	98871.444	41781.28	0.4225818730835973
M_ID_5db0c74036	New Look Wholesale	172004.022	51682.51999999999	0.3004727412711314
M_ID_80e51390c6	Kerby's Retailer	220467.474	61052.4	0.27692248154482874
M_ID_3111c6df35	Biltmore Menswear	1177293.6179999996	240143.76	0.2039794969822049
M_ID_c9855bf3e3	Druxy Boots	337256.24399999983	65086.34	0.1929877983222752
M_ID_b9f9332438	Fairbank Direct	291553.47	54451.16	0.1867621743620476
M_ID_9e84cda3b1	Indicohosts	226692.2880000002	41945.92	0.18503461396975251
M_ID_445742726b	Blvd Media	952645.3200000001	157761.69	0.16560380520212914
M_ID_9dbbaabb0a	Payless ShoeSource	140378.54400000014	23088.10000000002	0.16447029112939068

Group by

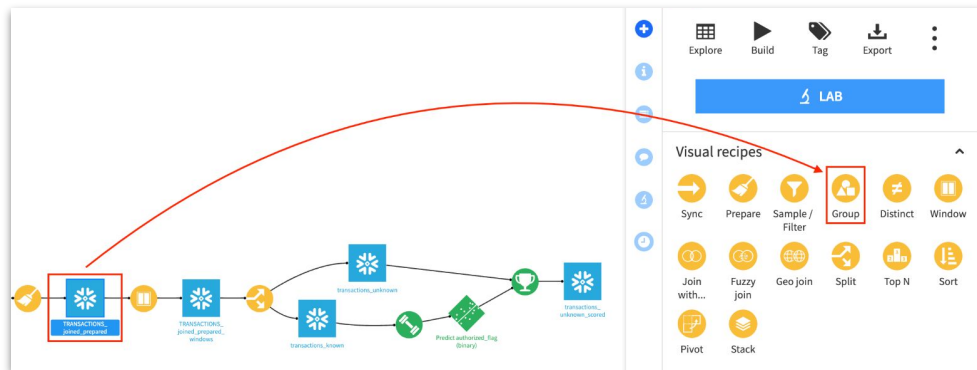


Obtain aggregated information from groups within rows in a dataset

Merchant Analysis

Group by merchant to retrieve fraudulent transactions

TO DO: Let's start by re-organizing our dataset at the merchant level.
Use a **Group** recipe by *merchant_id*.



New group recipe

Input dataset	Output dataset
Input dataset TRANSACTIONS_joined_prepared DATASET - View	Name fraud_transactions_by_merchant_id
Group By merchant_id	Store into managed-snowflake
Additional grouping keys can be added later.	
NEW DATASET USE EXISTING DATASET	
CANCEL CREATE RECIPE	

Merchant Analysis

Group by merchant to retrieve fraudulent transactions

TO DO: Set the **pre-filter** to retrieve only the **fraudulent transactions**.

The screenshot displays the Dataiku interface for a workflow named 'compute_fraud_transactions_by_merchant_id'. The workflow is visualized on the left as a vertical sequence of steps: 'Pre-filter' (highlighted with a red box and a green checkmark), 'Computed columns', 'Group' (with a green checkmark), 'Custom aggregations', and 'Post-filter'. The main panel shows the configuration for the 'Pre-filter' step. It includes a 'Distinct rows' section with a toggle set to 'OFF' and a 'Filter' section with a toggle set to 'ON'. The filter configuration is set to 'Keep only rows that satisfy the following conditions'. A single condition is defined: 'Where authorized_flag == 0', which is also highlighted with a red box. Below the condition, there is a link to '+ ADD A CONDITION'. The top navigation bar includes tabs for 'Settings', 'Input / Output', 'Advanced', and 'History', along with 'SAVE' and 'ACTIONS' buttons.

Merchant Analysis

Group by merchant to retrieve fraudulent transactions

TO DO: Retrieve **sum of purchase_amount** and some **descriptive information about each merchant**.

compute_fraud_transactions_by_merchant_id

Settings Input / Output Advanced History SAVE ACTIONS

Pre-filter

Computed columns

Group

Custom aggregations

Post-filter

Group Keys

Create a group for each unique combination of these variables

merchant_id string

Select key to add ADD or create a new computed column

☐ Compute count for each group

Per field aggregations Select Aggregations to be computed for each field

0 / 32 Filter column... Hide unused variables

☐ merchant_category_id	bigint	Distinct	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼
☐ purchase_amount	double	Distinct	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼
☐ merchant_subsector_description	string	Distinct	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼
☐ merchant_geopoint	+ GROUP KEY string	Distinct	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼
☐ merchant_state	string	Distinct	Min	Max	Avg	Sum	Std. dev.	Count	First	Last	Concat	options ▼

Merchant Analysis

Group by merchant to retrieve fraudulent transactions

TO DO: For convenience you can rename the output columns.

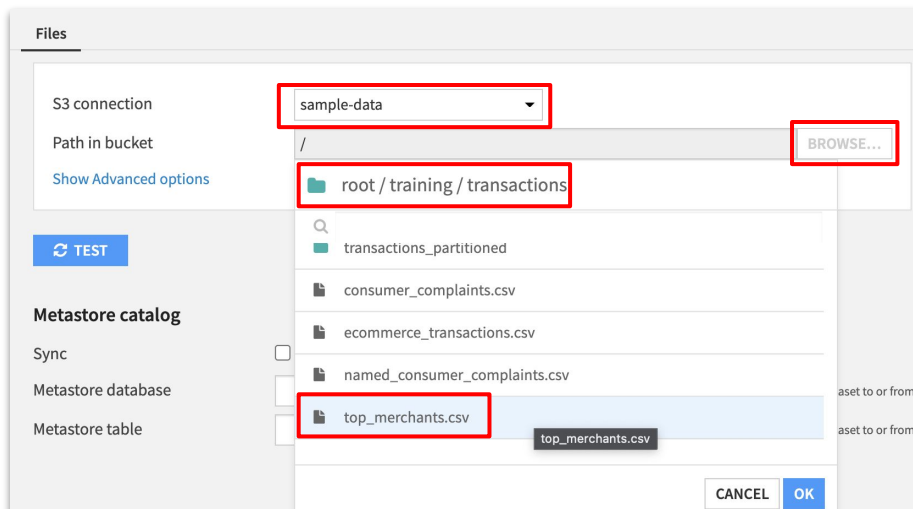
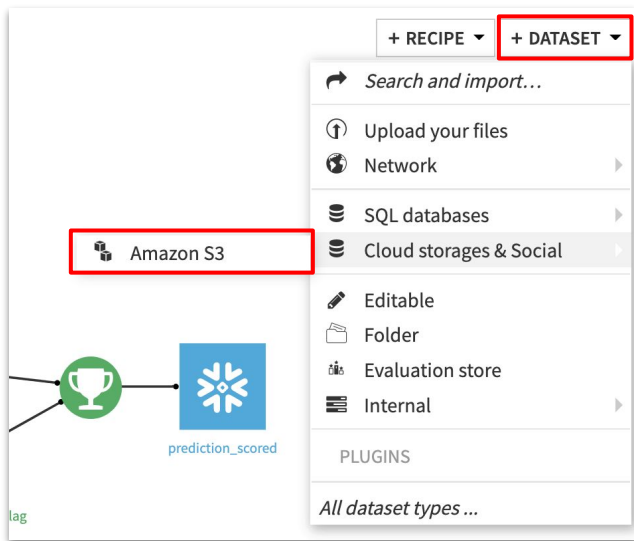
The screenshot displays a data processing interface. On the left, a vertical flowchart shows the steps: Pre-filter, Computed columns, Group, Custom aggregations, Post-filter, and Output. The 'Output' step is highlighted with a red box and a green checkmark. The main panel shows the execution method as 'In-database (SQL)' with options to 'VIEW QUERY' or 'CONVERT TO SQL RECIPE'. Below this, a table titled 'Output column names' lists six columns with their data types and original names, each with a 'RESET' button.

Index	Data Type	Column Name	Action	Original Column Name
1	(string)	merchant_id	RESET	
2	(bigint)	merchant_category	RESET	original column name 'merchant_category_id_first'
3	(double)	sum_fraud_transactions	RESET	original column name 'purchase_amount_sum'
4	(string)	merchant_subsector_description	RESET	original column name 'merchant_subsector_description_first'
5	(string)	merchant_geopoint	RESET	original column name 'merchant_geopoint_first'
6	(string)	merchant_state	RESET	original column name 'merchant_state_first'

Merchant Analysis

Add a dataset from a S3 bucket

TO DO: We have at our disposal an S3 bucket. Let's retrieve the dataset *top_merchant* from it. It contains the 100 merchants with the highest revenue in our database. Check out the dataset preview.



- ✓ Used `/top_merchants.csv` (5.18 KB) to parse data
- ✓ Used format csv and found 4 columns

PREVIEW >

Merchant Analysis

Add a dataset from a S3 bucket

TO DO: In the dataset preview, head to the “Schema” tab, and **Infer types from data**. Then **Create** the dataset.

New Amazon S3 Dataset

New dataset name: top_merchants_1 CREATE

Files Format / Preview **Schema** Partitioning Advanced

Columns 0 / 4 All types All meanings + ADD COLUMN **INFER TYPES FROM DATA** EXPORT SCHEMA

☐	1	merchant_id	string	Auto-detect
☐	2	merchant_name	string	Auto-detect
☐	3	merchant_category_id	string	Auto-detect
☐	4	merchant_turnover	string	Auto-detect

This option allows Dataiku to assign new storage types to the dataset, in line with the actual content of the data.

Infer storage types from data

This only takes into account a very small sample of data, and could lead to invalid data. For safer typing of data, use a prepare recipe.

CANCEL

CONFIRM

New dataset name: top_merchants_1

CREATE

Merchant Analysis

Join top merchants

TO DO: Join the datasets **top_merchants** and **fraud_transactions_by_merchant_id**.

New join recipe

Input datasets

top_merchants
DATASET - View

fraud_transactions_by_merchant_id
DATASET - View

Additional inputs can be added after the recipe creation.

Output dataset

Name
top_merchants_joined

Store into
managed-snowflake

NEW DATASET | USE EXISTING DATASET

CANCEL

CREATE RECIPE

Merchant Analysis

Create a post-join computed column

TO DO: Keep **default settings** and create a **post-join computed column**. The formula will let us compute the percentage of each merchant's turnover that was identified as fraudulent.

Should be orange with
**warning: Recipe cannot use
in-database engine. DSS
engine (slow) will be used**

The screenshot shows the Dataiku interface for the 'compute_top_merchants_joined' recipe. The 'Join' step is highlighted with a red circle and a red arrow pointing to the warning text. The 'Post-join computed columns' section is visible, showing the formula 'sum_fraudulent_transactions/merchant_turnover' and a preview table.

Computed columns

You can create new columns using SQL or DSS formulas.

Name: percent_fraud Store as: double Mode: DSS formula

Columns

Columns	Variables
merchant_id	string
sum_fraudulent_transactio...	double
merchant_subsector_descri...	string
merchant_location	string
merchant_state	string

Preview

percent_fraud	sum_fraudulent_transactions	merchant_turnover
	15.62	
	182.7800000000000003	
	30524.409999999996	
	392.07	

Preview weird because of sampling

Merchant Analysis

Output dataset: top_merchants_joined

Joined dataset has new merchant info columns: merchant_name & merchant_turnover

top_merchants_joined 100 rows

merchant_id	merchant_name	merchant_category_id	merchant_turnover	purchase_amount_s...	merchant_subsector_description	merchant_location	merchant_state	count	percent_fraud
string Text	string Text	bigint Integer	double Decimal	double Decimal	string Text	string GeoPoint	string US State	bigint Integer	double Decimal
M_ID_e4129b5119	My Own Retail Store	119	181517.094	13989.21	insurance	POINT(-70.428 44.88)	Maine	27	0.07706827875946493
M_ID_42846e7bc8	Nautical Pail	796	175238.94600000008	10394.86	insurance	POINT(-73.779 42.676)	New York	24	0.059318206581772046
M_ID_c1a7024db1	Shedd Knives	171	107070.04800000001	5988.1399999999999	gas	POINT(-72.655 41.795)	Connecticut	16	0.05592731218351559
M_ID_8fcb16a959	Ipswich Grocers	119	223511.47199999995	9202.45	insurance	POINT(-73.96 41.699)	New York	16	0.041172159610670916
M_ID_59764e8cb1	Cape Whitsundays	755	560456.784	45666.299999999996	internet	POINT(-70.23 44.613)	Maine	45	0.0814805018043996
M_ID_2637773dd2	Droghedale	422	348931.70999999973	10276.310000000001	consumer electronics	POINT(-71.544 43.226)	New Hampshire	54	0.02945077705892654
M_ID_a9d91682ad	Eskimo Wholesale	506	314510.94000000001	11374.869999999999	dental	POINT(-72.429 43.578)	Vermont	49	0.03616685003071751
M_ID_9ba1cf53bc	Globe Outlet	333	276772.896	20095.63	insurance	POINT(-79.964 40.392)	Pennsylvania	22	0.07260692896749543
M_ID_3f3084da59	Store SGS	367	99052.937999999994	2379.85	carpentry	POINT(-71.55 43.182)	New Hampshire	10	0.024026041509238236
M_ID_f416f2710c	Sammi's Outlet Store	171	111804.46200000001	4591.42	gas	POINT(-73.788 42.683)	New York	21	0.041066518436446625
M_ID_4e4617e14	Kangaroo Lounge	333	221125.734	11821.43	insurance	POINT(-73.78 42.61)	New York	14	0.0534602182485011
M_ID_3700a4ec5f	Spirit & Splendid	298	100735.938	11113.07	insurance	POINT(-73.883 41.711)	New York	13	0.11031882186871582
M_ID_b9dc28cb9	Exhibition Stores	401	302594.22000000006	30524.41	luxury goods	POINT(-73.974 41.733)	New York	88	0.10087572062678507
M_ID_01f4e277b0	Wootton Retail	299	87102.0	48390.0	luxury goods	POINT(-72.844 41.532)	Connecticut	1	0.5555555555555556
M_ID_aa91fc5309	Whispers Outlet	705	87151.176	3383.1800000000003	groceries	POINT(-73.805 42.627)	New York	14	0.03881967123426998
M_ID_fc7d7969c3	Newcastle Direct	401	171924.46200000617	16864.48	luxury goods	POINT(-71.765 42.217)	Massachusetts	686	0.09809238199040805
M_ID_b9f9332438	Fairbank Direct	357	291553.47	53138.79	consumer electronics	POINT(-71.534 43.219)	New Hampshire	69	0.1822608731084559
M_ID_ba1ce2074b	Ziksilver	87	85984.84799999995	3767.1	consumer electronics	POINT(-73.752 42.669)	New York	19	0.04381120729549934
M_ID_7064aa8a68	Proclaim Retail Shop	419	130773.99600000003	4514.1299999999999	biotechnology	POINT(-73.748 42.672)	New York	10	0.03451855979074003
M_ID_445742726b	Blvd Media	119	952645.32000000001	144360.47	insurance	POINT(-71.813 42.241)	Massachusetts	278	0.1515364291087894
M_ID_c8f44ec011	Leithy Retail	333	203977.31399999999	6442.13	insurance	POINT(-73.894 41.693)	New York	10	0.031582580796215426
M_ID_3111c6df35	Biltmore Menswear	119	1177293.6179999996	218535.96000000002	insurance	POINT(-79.982 40.413)	Pennsylvania	226	0.18562570684044946

Merchant Analysis

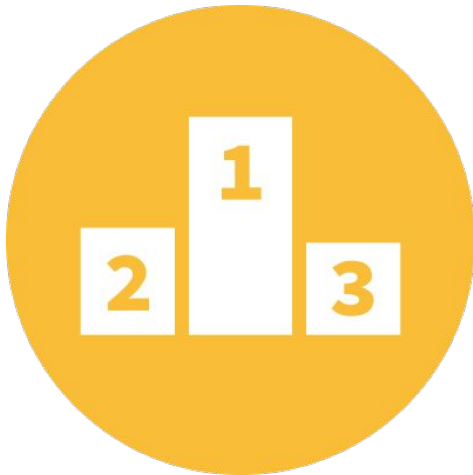
Filter only merchants with the highest fraud percentage

Let's identify the **Top 10 merchants** who have registered the **highest amount of fraud**.



merchant_id	merchant_name	merchant_turnover	Sum_fraudulent_transactions	percent_fraud
string Text	string Text	double Decimal	double Decimal	double Decimal
M_ID_01f4e277b0	Wootton Retail	87102.0	48390.0	0.5555555555555556
M_ID_9f227ee53d	Sunset Imports	98871.444	41781.28	0.4225818730835973
M_ID_5db0c74036	New Look Wholesale	172004.022	51682.51999999999	0.3004727412711314
M_ID_80e51390c6	Kerby's Retailer	220467.474	61052.4	0.27692248154482874
M_ID_3111c6df35	Biltmore Menswear	1177293.6179999996	240143.76	0.2039794969822049
M_ID_c9855bf3e3	Druxy Boots	337256.24399999983	65086.34	0.1929877983222752
M_ID_b9f9332438	Fairbank Direct	291553.47	54451.16	0.1867621743620476
M_ID_9e84cda3b1	Indicohosts	226692.2880000002	41945.92	0.18503461396975251
M_ID_445742726b	Blvd Media	952645.3200000001	157761.69	0.16560380520212914
M_ID_9dbbaabb0a	Payless ShoeSource	140378.54400000014	23088.10000000002	0.16447029112939068

Top N



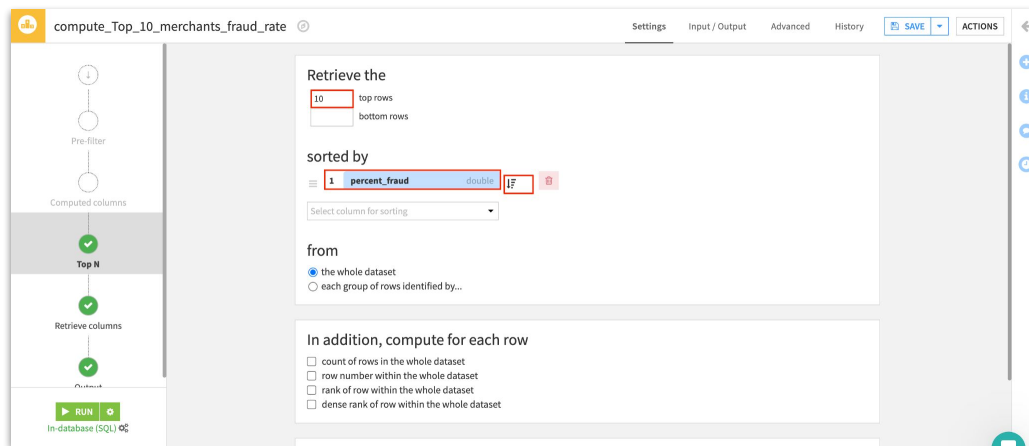
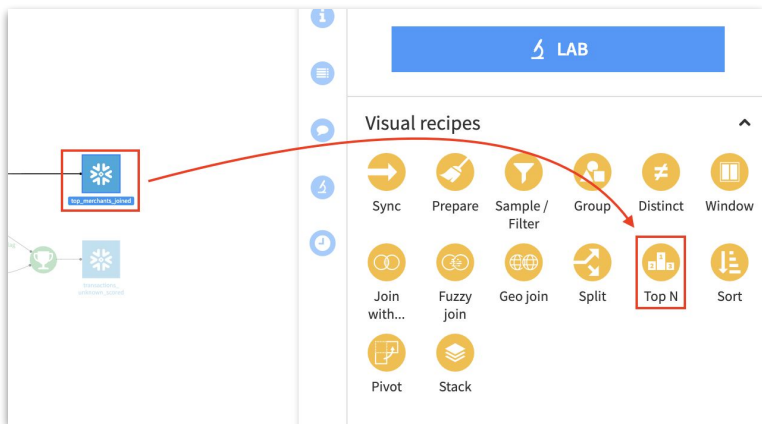
The Top N recipe allows you to filter a dataset based on the top and/or bottom values of some of its rows.

Merchant Analysis

Identify the 10 merchants with the highest fraud percentage

TO DO: Use a Top N recipe to get the 10 rows with the highest *percent_fraud*.

Hint : Make sure you set the sort order as descending by clicking on the sorting icon next to *percent_fraud*.



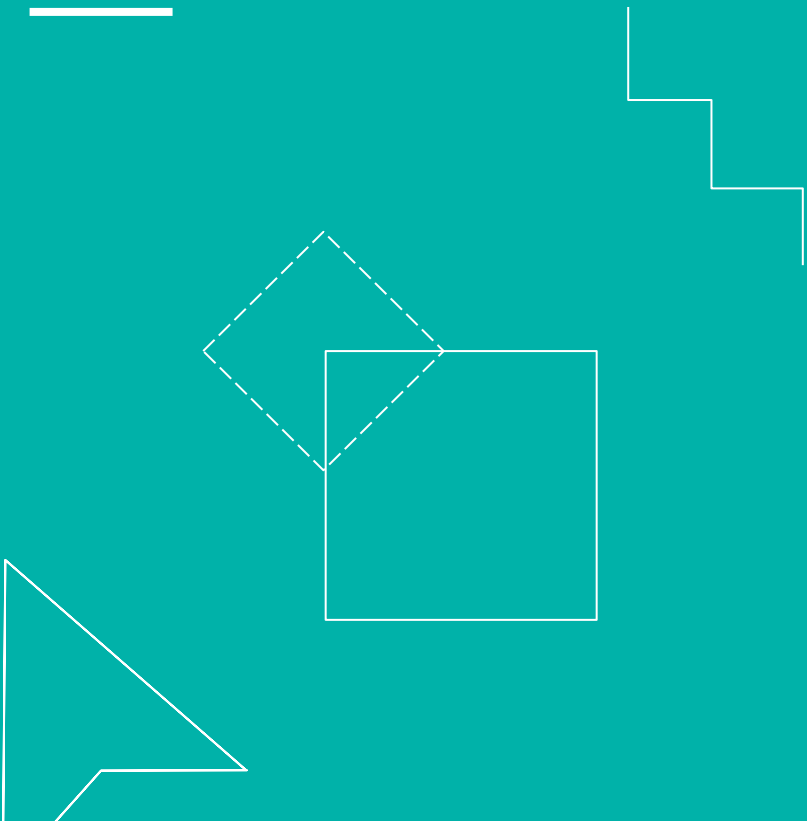
Merchant Analysis

Identify the 10 merchants with the highest fraud percentage

TO DO: Congrats ! We have identified the top 10 merchants who have registered the most fraud, as a percent of their total turnover.

merchant_id	merchant_name	merchant_category_id	merchant_turnover	sum_fraud_amount	percent_fraud
string Text	string Text	bigint Integer	double Decimal	double Decimal	double Decimal
M_ID_01f4e277b0	Wootton Retail	299	87102.0	48390.0	0.5555555555555556
M_ID_9f227ee53d	Sunset Imports	63	98871.444	41781.28	0.4225818730835973
M_ID_5db0c74036	New Look Wholesale	171	172004.022	51682.52	0.30047274127113144
M_ID_80e51390c6	Kerby's Retailer	416	220467.474	61052.4	0.27692248154482874
M_ID_3111c6df35	Biltmore Menswear	119	1177293.6179999996	240143.76	0.2039794969822049
M_ID_c9855bf3e3	Druxy Boots	299	337256.24399999983	65086.34	0.1929877983222752
M_ID_b9f9332438	Fairbank Direct	357	291553.47	54451.16	0.1867621743620476
M_ID_9e84cda3b1	Indicohosts	884	226692.2880000002	41945.92	0.18503461396975251
M_ID_445742726b	Blvd Media	119	952645.3200000001	157761.69	0.16560380520212914
M_ID_9dbbaabb0a	Payless ShoeSource	130	140378.54400000014	23088.1	0.16447029112939066

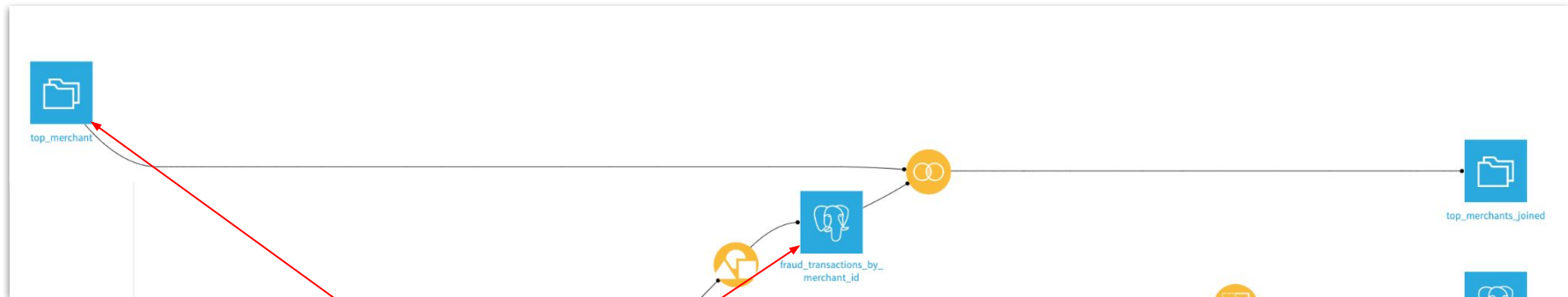
AGENDA

Abstract geometric shapes on a teal background, including a white horizontal line, a white stepped line, a white dashed diamond, a white solid square, and a white triangle.

1. Background and Objectives
2. Modifying the Flow
 - a. Window recipe
 - b. Schema Propagation
3. Merchants analysis
 - a. Group recipe
 - b. TopN recipe
 - c. Pushdown computation**
4. Dataiku Applications
 - a. Project variables
 - b. Scenarios
 - c. Application Designer

Merchant Analysis

Pushdown Computation : What's wrong with the join recipe?



(Hint: Input Datasets)

The datasets are not located on the same database !

Hence, DSS will use the “streaming engine” to perform the join... and that makes DSS sad :(

Merchant Analysis

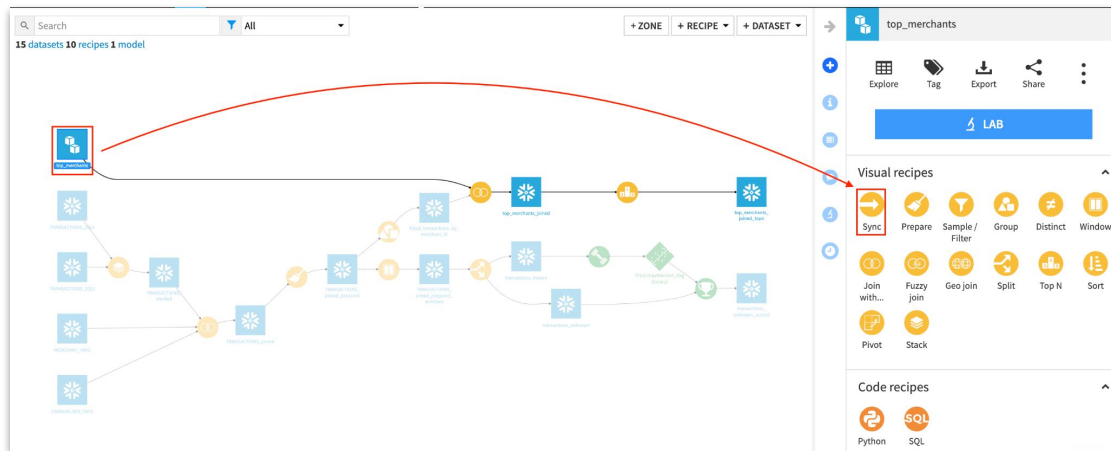
Pushdown Computation : What, Why, Where and How

- DSS will always choose the **best computation option** available for the execution of recipes.
- Computation can either be done on DSS or be pushed down towards the data sources, i.e. a Snowflake database, Hadoop cluster, etc.
- Useful to prevent issues involving computation and lack of memory especially when dealing with huge datasets.

Merchant Analysis

Pushdown Computation : Sync the “top_merchant” Dataset to SQL database

TO DO: to ensure all the input datasets to the Join Recipe are stored on the same database, we can use a Sync Recipe on the *top_merchant* dataset. Create and Run the recipe.



New sync recipe

Input dataset	Output dataset
top_merchants DATASET - View	Name top_merchants_sql
	Store into managed-snowflake
NEW DATASET USE EXISTING DATASET	
CANCEL CREATE RECIPE	

Merchant Analysis

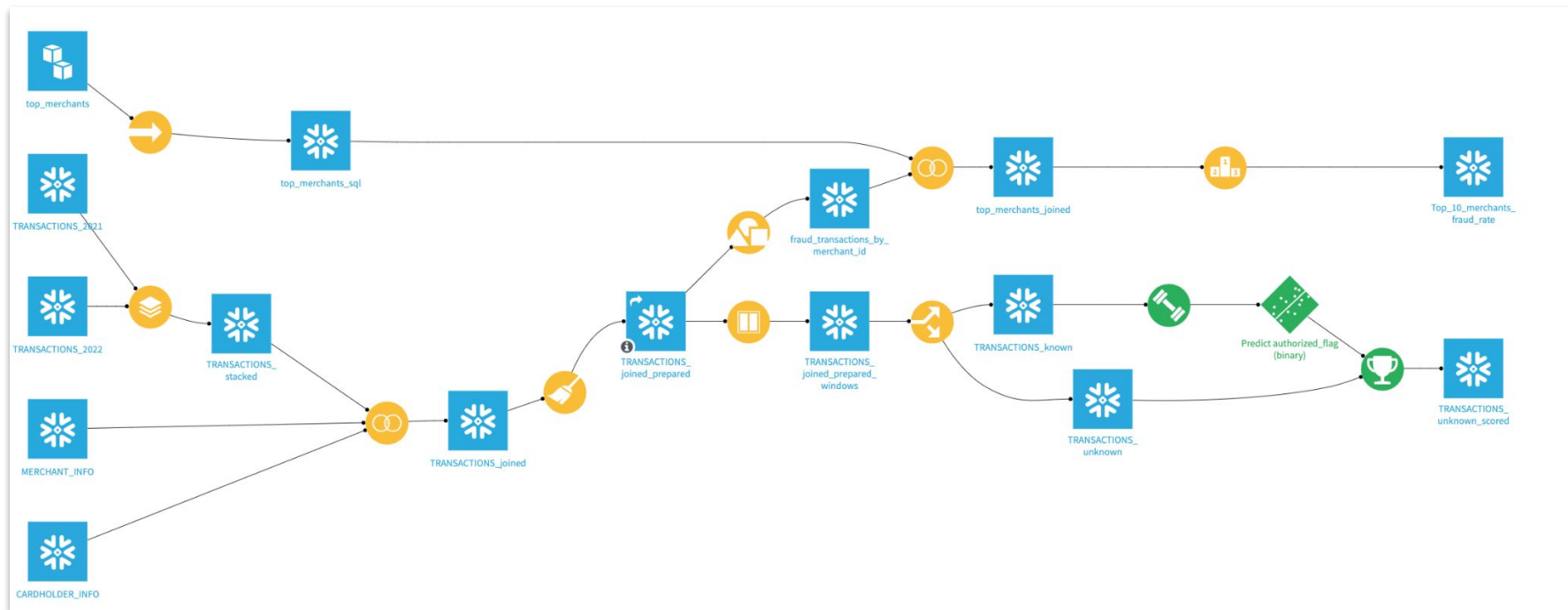
Pushdown Computation : Remap the Input of the Join Recipe

TO DO: now, from within the Join recipe, replace the *top_merchants* dataset for the *top_merchants_sql* dataset.

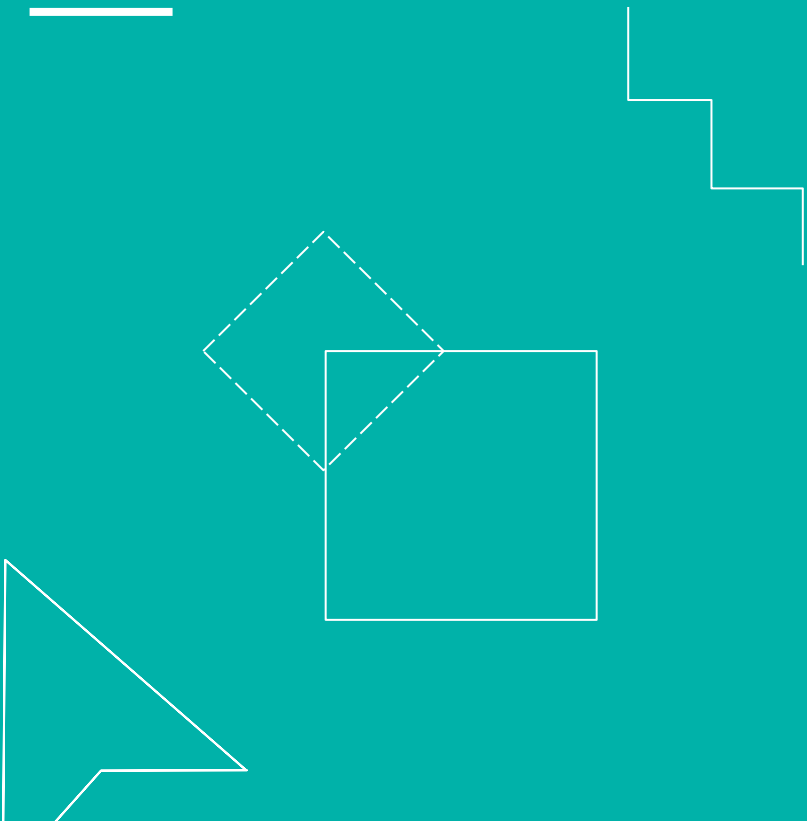
The screenshot displays the Dataiku interface for a recipe named "compute_top_merchants_joined". The recipe is currently in the "Join" step, which is a LEFT JOIN of two datasets: "top_merchant" and "fraud_transactions_by_merchan...". The "top_merchant" dataset is highlighted with a red box, and a tooltip indicates "click to replace dataset". A modal dialog titled "Replace recipe input" is open, showing the "top_merchant_sql" dataset selected for replacement. The dialog also shows a green checkmark indicating "This dataset can be used". At the bottom right, there is a "RUN" button and a note "In-database (SQL)".

Merchant Analysis

Flow Check: Your flow should look like this



AGENDA



1. Background and Objectives
2. Modifying the Flow
 - a. Window recipe
 - b. Schema Propagation
3. Merchants analysis
 - a. Group recipe
 - b. TopN recipe
 - c. Pushdown computation
4. Dataiku Applications
 - a. **Project variables**
 - b. Scenarios
 - c. Application Designer

Project Variables

Concept

In programming, variables store pieces of information, usually with the goal of reusability in mind; Variables allow for more dynamic programming and reduce the need to hard code values repeatedly.

You can use variables:

- In all Dataiku DSS visual recipes
- Code recipes
- Scenarios
- Other objects in the Flow.

Project Variables

Context

What if we want to compute the top 10 merchant for a specific US state ?

We could add a Pre-filter in the Group recipe with the state we have in mind.

Filter

Keep only rows that satisfy all the following conditions

authorized_flag	==	0	×
merchant_state	equals	Pennsylvania	×

+ ADD A CONDITION

RUN

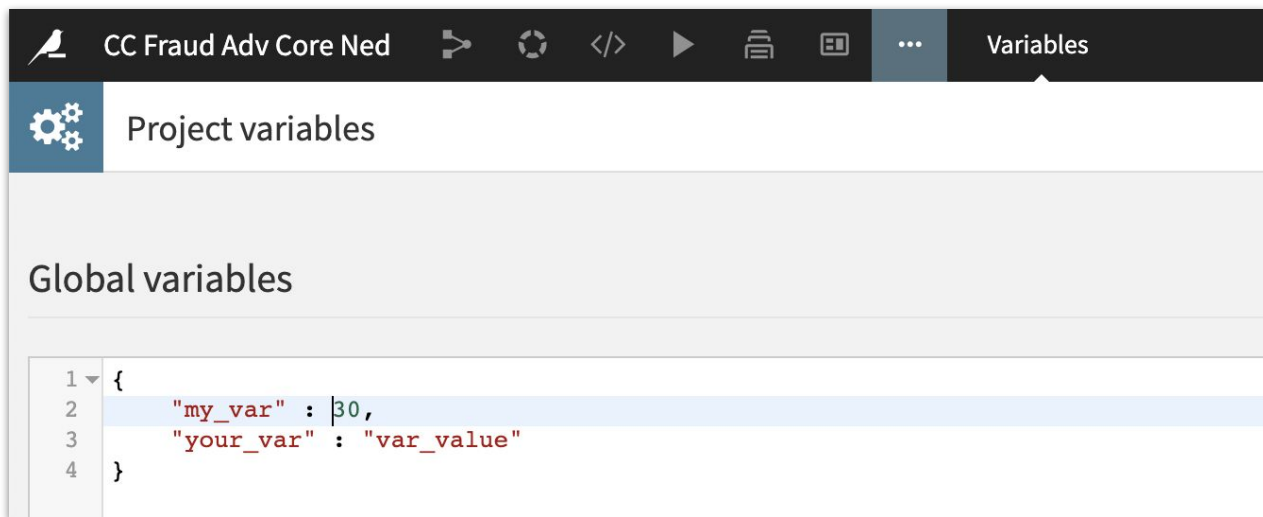
In-database (SQL)

Project Variables

Setting DSS Project Variables

DSS Project variables can be accessed via the *SETTINGS* → *Variables* page of a project.

- They must be defined in json format
- Double quotes should be used for variable names and strings



The screenshot shows the 'Variables' tab in the DSS interface for a project named 'CC Fraud Adv Core Ned'. The 'Project variables' section is active, displaying 'Global variables' in a code editor. The code is a JSON object with two entries: 'my_var' with a value of 30, and 'your_var' with a value of 'var_value'. The code is as follows:

```
1 {  
2   "my_var" : 30,  
3   "your_var" : "var_value"  
4 }
```


Project Variables

Accessing DSS Project Variables

- DSS Formula Language (e.g. in a Prepare recipe and various configuration fields)

```
if(Country == '${Country}', 1, 0)  
if(length_tweet < ${length_min}, 'small', 'big')
```

- SQL/PIG/HIVE/IMPALA/SparkSQL:

```
SELECT * from mytable WHERE condition='${VARIABLE_NAME}';
```

- Python

```
import dataiku  
variable = dataiku.get_custom_variables()['VARIABLE_NAME']  
  
print dataiku.get_custom_variables()  
{u'city': u'Paris', u'dip.home': u'/home/dataiku/dss', u'projectKey': u'WESTNILE_ADVANCED_DESIGN'}
```

Project Variables

Refactor the “US State” Filter

TO DO: first, we'll create a DSS variable called “state_name”, and assign it the value “New York”. **Remember to save before returning to the flow!**

The screenshot shows the DSS (Data Science Studio) interface. The top bar indicates the project is '[Sylvain] CC Fraud Adv Core'. The 'Variables' panel is open, showing 'Project variables' and 'Global variables'. Under 'Global variables', a JSON object is defined:

```
1 {  
2   "state_name": "New York"  
3 }
```

A dropdown menu is open, showing options: API Designer, Bundles, Variables, Macros, Version Control, Application Designer, Settings, and Security. The 'Variables' option is highlighted.

It's JSON format, so make sure you use **double quotes :)*

Project Variables

Refactor the “US State” Filter

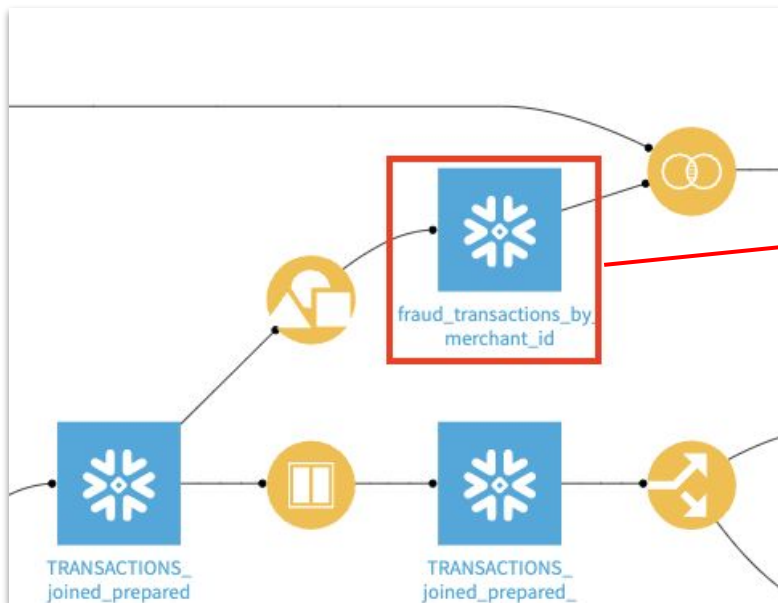
TO DO: now, return to the Group By recipe, and replace the merchant state filter value with `${state_name}`.
Re-run the recipe downstream.

The screenshot displays the Dataiku interface for a recipe named `compute_fraud_transactions_by_merchant_id`. The interface includes tabs for Summary, Settings, Input / Output, Advanced, and History, along with buttons for SAVE and ACTIONS. The recipe flow on the left shows a Pre-filter step (highlighted with a red box) followed by a Group step. The main configuration area is for the Filter step, which is currently set to "Distinct rows" (with a toggle for "Duplicate rows are allowed" set to OFF). The Filter configuration is set to "Keep only rows that satisfy all the following conditions". Two conditions are listed: `authorized_flag == 0` and `merchant_state equals ${state_name}`. The second condition is highlighted with a red box. A "+ ADD A CONDITION" button is visible below the conditions.

Project Variables

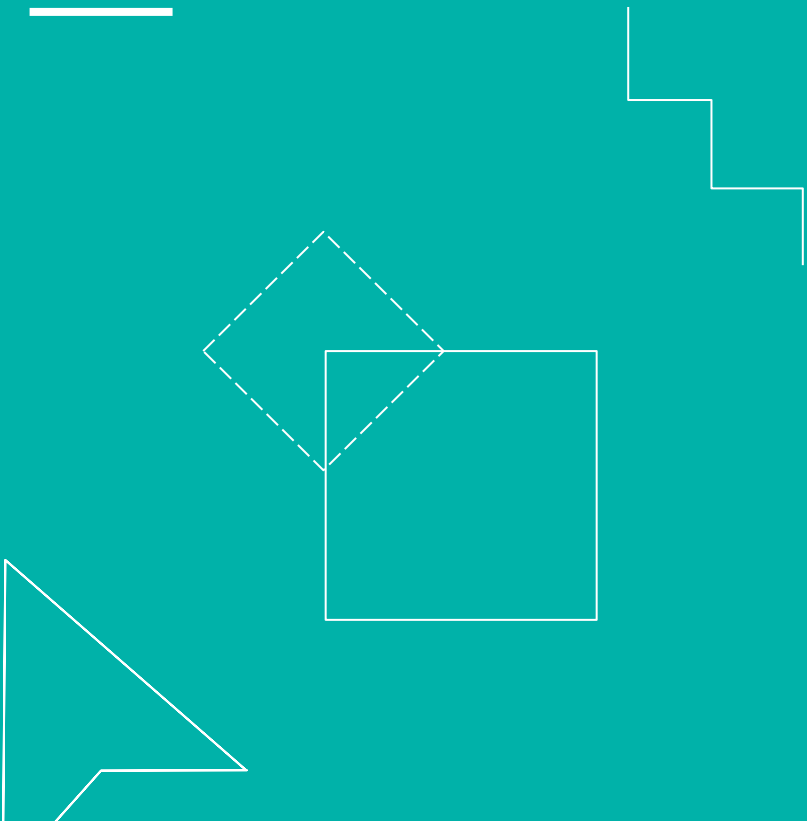
Refactor the “US State” Filter

TO DO: finally, to ensure the filter worked as expected, navigate to the *fraud_transactions_by_merchant_id* dataset to check that all merchants are from New York .



	merchant_subsector_descrip...	merchant_location	merchant_state
	string Text	string GeoPoint	string US State
7	consumer electronics	POINT(-73.758 42.646)	New York
5	luxury goods	POINT(-73.924 41.696)	New York
6	consumer electronics	POINT(-73.959 41.665)	New York
3	consumer electronics	POINT(-73.782 42.678)	New York
2	insurance	POINT(-73.716 42.63)	New York
4	consumer electronics	POINT(-73.927 41.729)	New York
4	consumer electronics	POINT(-73.79 42.688)	New York
1	consumer electronics	POINT(-73.974 41.711)	New York
9	gas	POINT(-73.739 42.624)	New York
4	groceries	POINT(-73.756 42.604)	New York

AGENDA

Abstract geometric shapes on a teal background, including a white horizontal line, a white stepped line, a white dashed diamond, a white solid square, and a white triangle.

1. Background and Objectives
2. Modifying the Flow
 - a. Window recipe
 - b. Schema Propagation
3. Merchants analysis
 - a. Group recipe
 - b. TopN recipe
 - c. Pushdown computation
4. Dataiku Applications
 - a. Project variables
 - b. Scenarios**
 - c. Application Designer

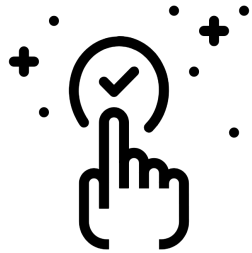
Scenarios



- Schedule actions to run automatically
- Can trigger on-demand, based on a predefined schedule, or depending on other conditions

Scenarios

Components



TRIGGERS



**SEQUENCE
OF STEPS**



**(OPTIONAL)
REPORTERS**

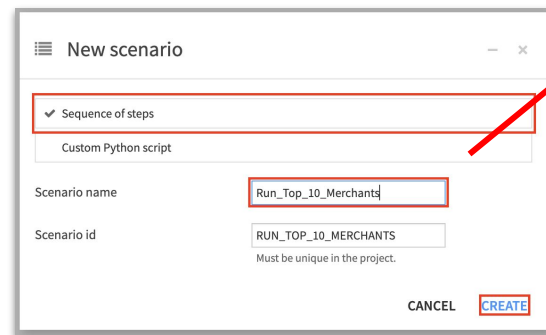
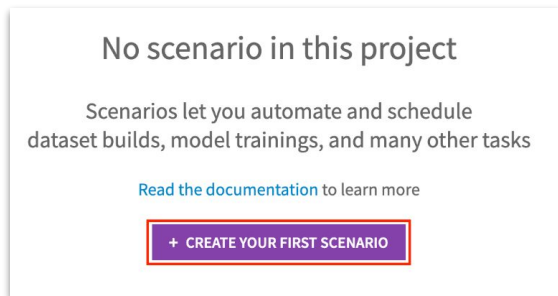
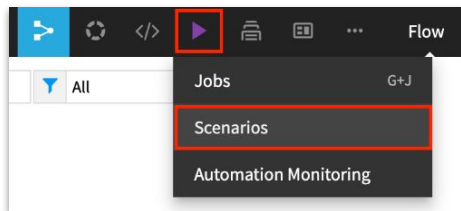
Scenarios

Create your scenario

TO DO: Create a new scenario

Choose the “sequence of steps” option

Note: advanced scenario customization are available with “custom python script” scenarios



Scenarios

Create your scenario

TO DO: Set-up a daily trigger for our scenario

Run

Auto-triggers ☐ OFF Run the scenario when a trigger activates.

Run as Last author of scenario

Triggers ADD TRIGGER

- Time-based trigger
- Trigger on dataset change
- Trigger on sql query change
- Trigger after scenario
- Custom trigger

Reporters

Add a Time-Based trigger

Triggers ADD TRIGGER

Time-based ☐ ON

Repeat every 1 days

Starting from 08/12/2022

Run at 00:00

Using timezone Server timezone

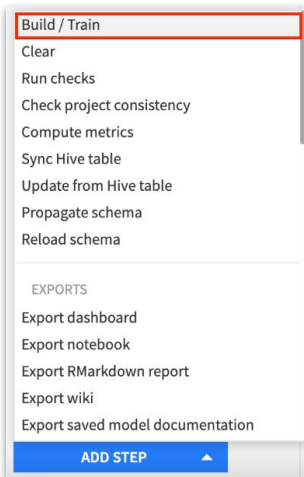
Set up the trigger

Scenarios

Build your scenario

TO DO: Go to the **Steps** tab to design the processes you'd like to automate.

Our first (and only) step will be a **Built / Train** step.



Build

This step lets you build one or several "computable" items of the Flow (build datasets, build managed folders, train saved models)
For partitioned datasets (and folders), you can use partitioning specs

Item	Partitions
	N/A

Build mode

Force-rebuild dataset and depend

Sync. Hive metastore

☒

Ignore failure

☐ Don't mark the scenario as failed, even if the build fails (warning: the datasets' states may be incoherent)

Run this step

If no prior step failed

Maximal number of retries

0

Number of additional attempts before considering the step definitively failed

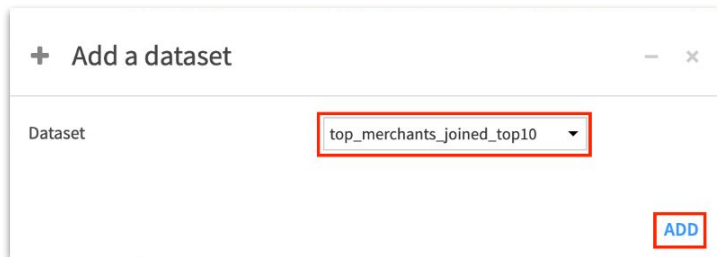
Scenarios

Build your scenario

TO DO: Add the `top_merchants_joined_top10` (or whatever your top10 dataset is named) dataset as the dataset to build.

⚠ Don't forget to change the build mode to **upstream** (so everything preceding it is also built) and **force-build** (so everything is rebuilt and not just those that get changed) and **check update output schemas** *.

Don't forget to save !

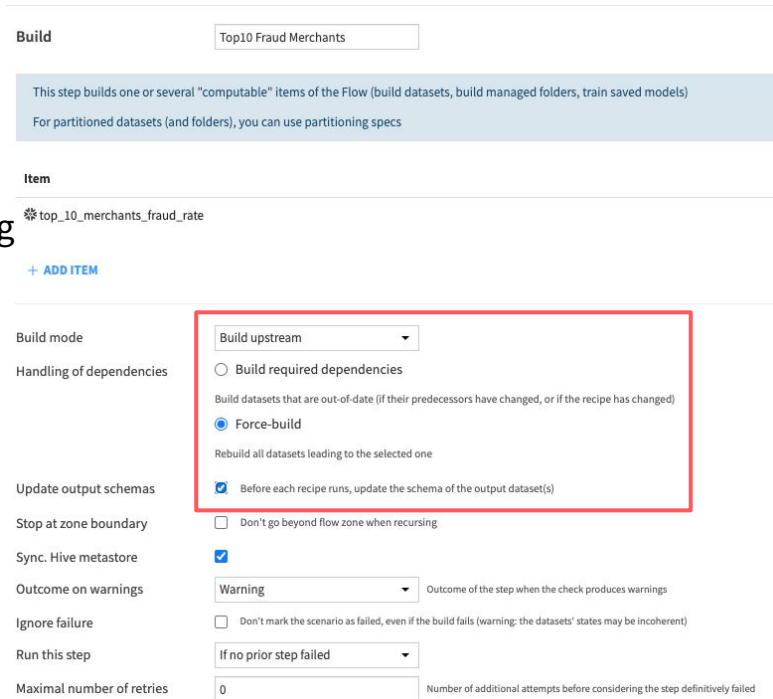


+ Add a dataset

Dataset: top_merchants_joined_top10

ADD

*We are using this [build mode](#) because we want to update our Flow after changing our filter variable ; in this context, DSS needs to be explicitly told to rebuild all the data, as it will not automatically detect the datasets as in need to be rebuilt.



Build: Top10 Fraud Merchants

This step builds one or several "computable" items of the Flow (build datasets, build managed folders, train saved models)
For partitioned datasets (and folders), you can use partitioning specs

Item: ⚙ top_10_merchants_fraud_rate

+ ADD ITEM

Build mode: Build upstream

Handling of dependencies:
☐ Build required dependencies
Build datasets that are out-of-date (if their predecessors have changed, or if the recipe has changed)
☒ Force-build
Rebuild all datasets leading to the selected one

Update output schemas:
☒ Before each recipe runs, update the schema of the output dataset(s)

Stop at zone boundary: ☐ Don't go beyond flow zone when recursing

Sync. Hive metastore: ☒

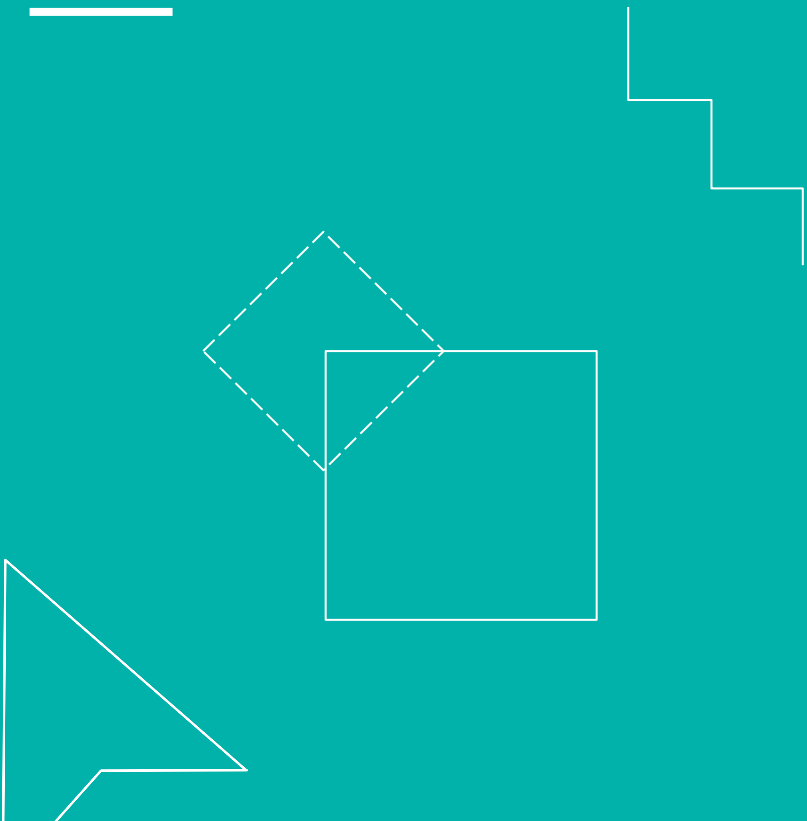
Outcome on warnings: Warning Outcome of the step when the check produces warnings

Ignore failure: ☐ Don't mark the scenario as failed, even if the build fails (warning: the datasets' states may be incoherent)

Run this step: If no prior step failed

Maximal number of retries: 0 Number of additional attempts before considering the step definitively failed

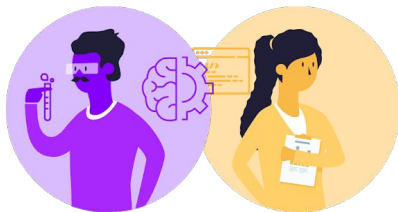
AGENDA

Abstract geometric shapes on a teal background, including a white horizontal line, a white stepped line, a white dashed diamond, a white solid square, and a white triangle.

1. Background and Objectives
2. Modifying the Flow
 - a. Window recipe
 - b. Schema Propagation
3. Merchants analysis
 - a. Group recipe
 - b. TopN recipe
 - c. Pushdown computation
4. Dataiku Applications
 - a. Project variables
 - b. Scenarios
 - c. **Application Designer**

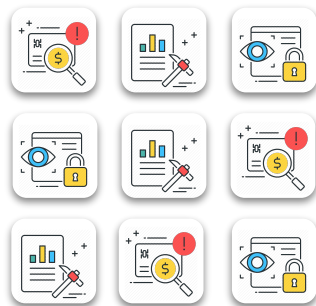
Dataiku Application

Introduction



**Data Scientists
& Data Analysts**

Design and package their projects
as reusable applications.



Dataiku Applications

Applications are available on
Dataiku's home



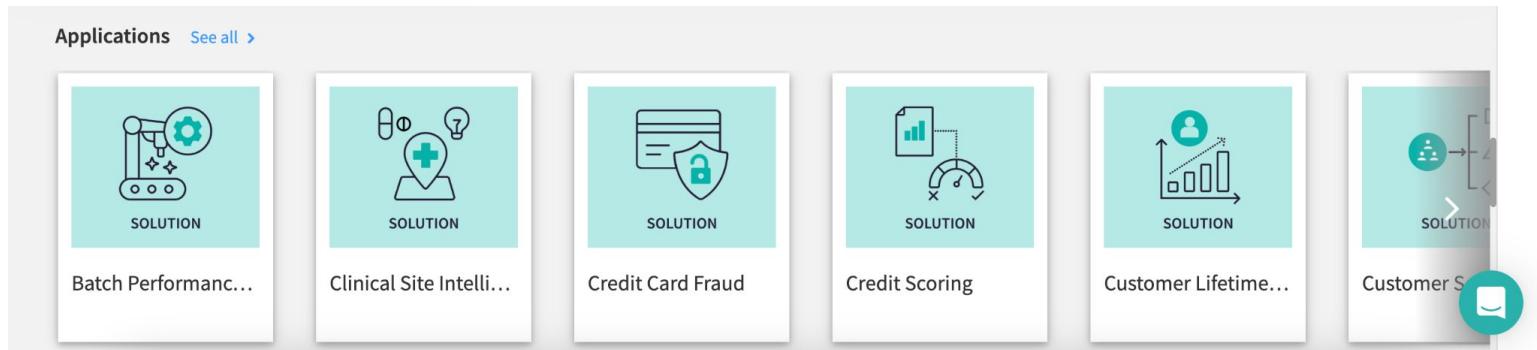
Business Analysts

Execute predefined tasks
& generate insights on the data
or parameters of their choices.



Dataiku Application

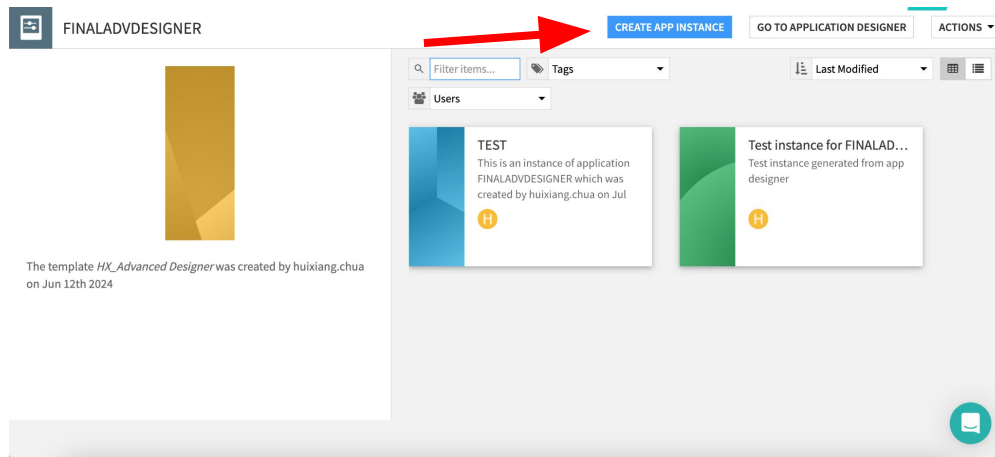
What are Applications?



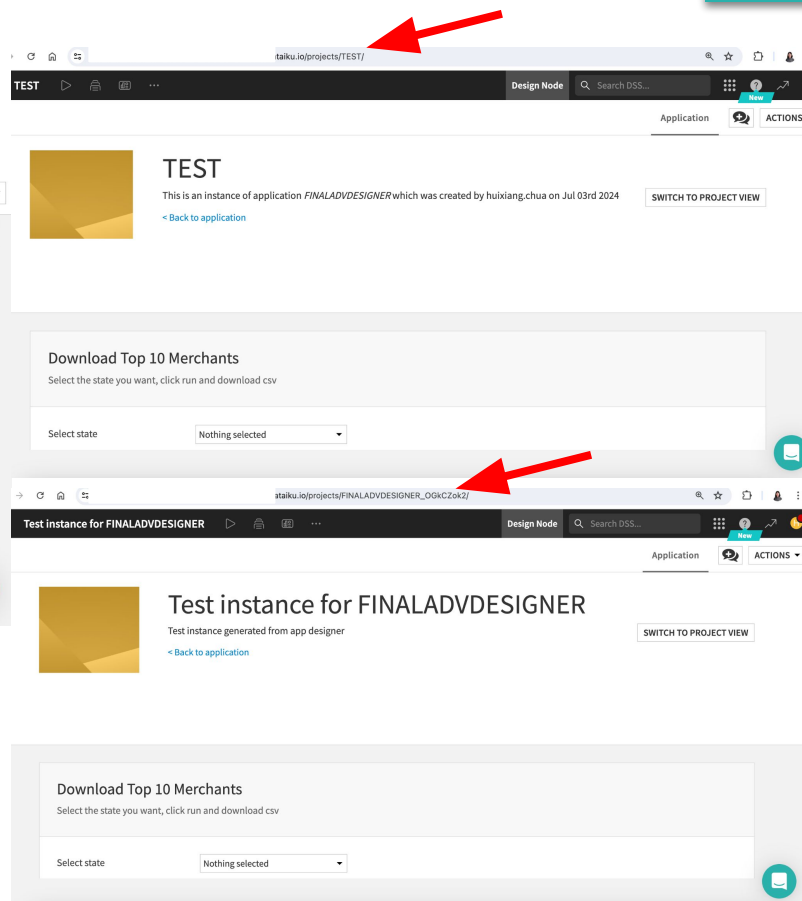
Users will be able to find applications that they have permissions to use under the Applications section on their Dataiku home page.

Dataiku Application

Creating an app instance (copy of a project)



Each individual will create an app instance for own usage which will create a copy of the original project. What they do will not overwrite anything in the flow in the original project. You can see that each app instance has their own unique URL (right).



Dataiku Application

Section Objective: Create a Dataiku Application

Objectives :

- Create a **DSS Application** that allows users to download the top 10 list of untrustworthy merchants of a given state.
- Use user input to modify project variables so that the state_name can be changed programmatically
- Use our **scenario** to update the flow automatically.



Test instance for FINALADVDESIGNER

Test instance generated from app designer

[< Back to application](#)SWITCH TO PROJECT VIEW

Download Top 10 Merchants

Input the state you want, click run and download the dataset

Select State	State Name	Nothing selected ▾
Run	RUN NOW	
Download	DOWNLOAD	

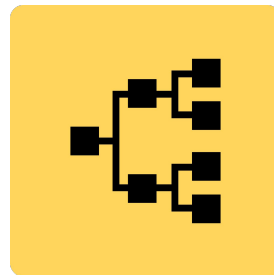
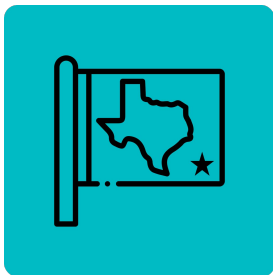
Dataiku Application

Game plan

2. Update state_name



1. Select a state



3. Build dataset

4. Download



Dataiku Application

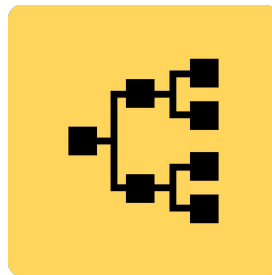
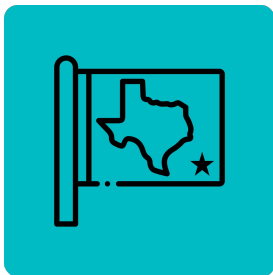
Game plan

2. Update state_name

4. Download



1. Select a state



3. Build dataset



↑
This step needs to be **automated**

Dataiku Application

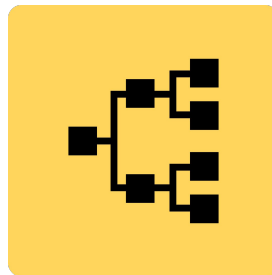
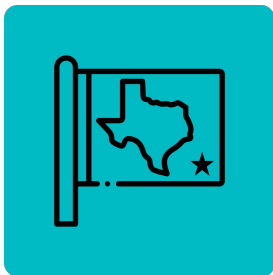
Game plan

2. Update state_name

4. Download



1. Select a state



3. Build dataset

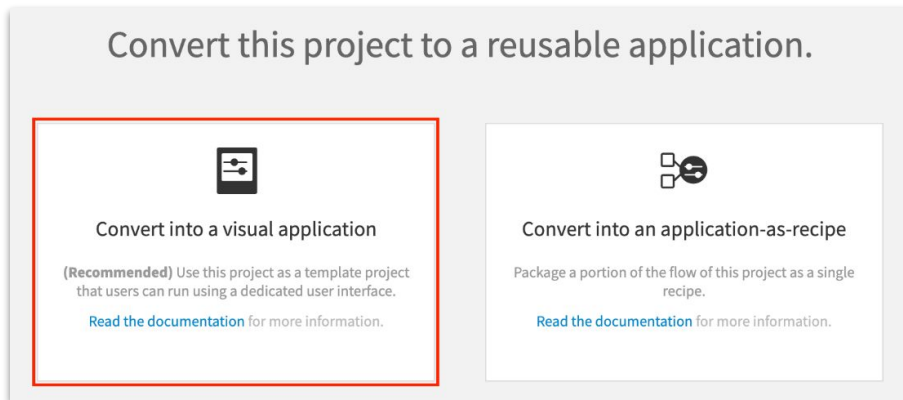
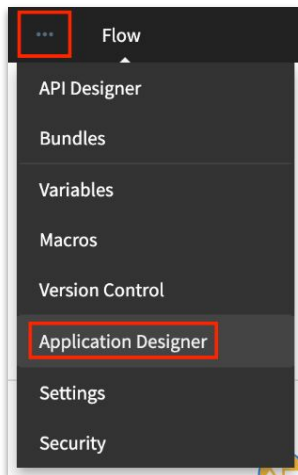


In order for our application to run, we will need to use our **scenario**, which the application we will create will be able to call !

Dataiku Application

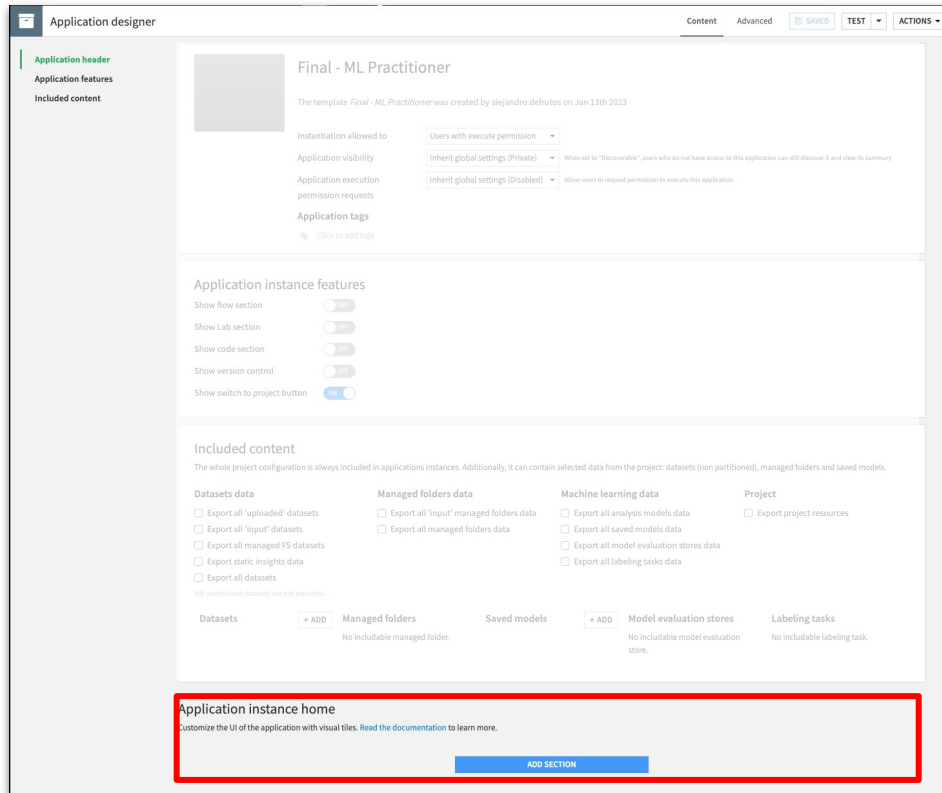
Create the app

Let's create our Dataiku Application. From the top navigation bar, **select the More Options menu (...)** > Application Designer, and select **Convert into a visual application**



Dataiku Application

Create the app



There are four main areas in the Application Designer interface :

- **Application header**
- **Application features**
- **Included content**
- **Application instance home**

Dataiku Application

Create the app

You don't need to change anything in the default parameters of the app.

Let's **add a section** to our application. This will be the landing page for your application. Go ahead and give it a name and description if you'd like.

Application instance home

Customize the UI of the application with visual tiles. [Read the documentation](#) to learn more.

ADD SECTION

Title

Download Top 10 Merchants

Description

Input the state you want, click run and download the dataset

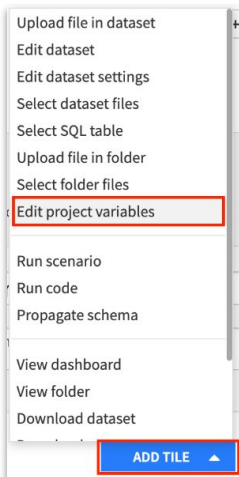
ADD TILE ▲

Dataiku Application

Create the app

Add a tile to the section, then select **Edit Project Variables** : this will allow the Application users to change a predefined project variable.

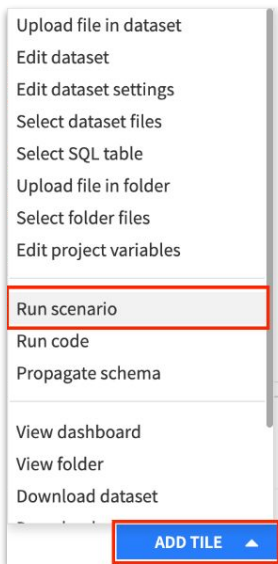
- Fill in the title of your tile
- Select Edit Inline With Auto-Save for the behaviour
- Fill in the Auto-generated controls section with the code snippet shared by the trainer (also in Wiki)



Dataiku Application

Create the app

Let's **add a second tile** to the application. This time, we will use the **Run Scenario** option, to allow our application to execute the scenario we created earlier.

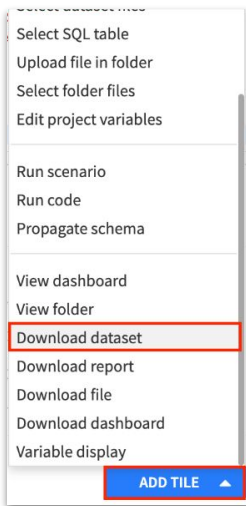


A screenshot of the 'Run scenario' configuration form. The form has a title 'Run scenario' at the top left. Below the title, there are three fields: 'Title' with the value 'Run', 'Scenario' with the value 'Run_Top_10_Merchants' (indicated by a dropdown arrow), and 'Button label' with the value 'Run now'. To the right of the 'Title' field, there is a blue link that says 'Add help'.

Dataiku Application

Create the app

Let's **add a third tile** to the application. Select **Download Dataset** : this will allow users to download the updated dataset our scenario will create.



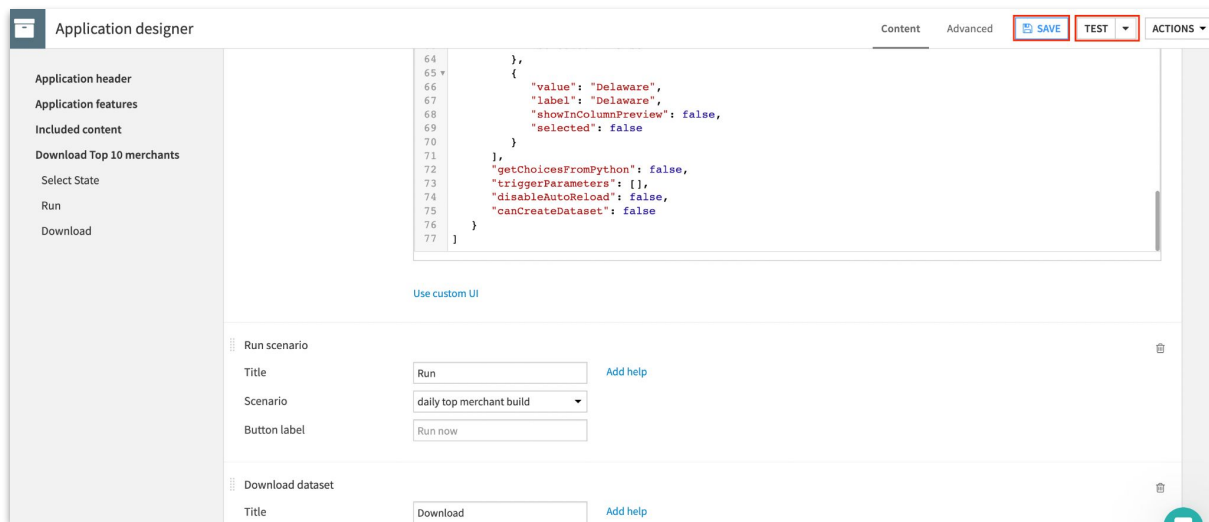
A screenshot of the 'Download dataset' configuration panel in Dataiku. The panel has a title 'Download dataset' and an 'Add help' link. It contains two main fields: 'Title' with a text input containing 'Download' (highlighted with a red box) and 'Dataset' with a dropdown menu showing 'top_merchants_joined_top10' (also highlighted with a red box). Below these fields are two tabs: 'Export to file' (active) and 'Other exports'. Under the 'Export to file' tab, there are three settings: 'Format' set to 'CSV (*.csv)', 'Separator' set to a comma character (with a tooltip that says 'Can be a single character, or a Unicode escape sequence like \u0001'), and 'With header' checked with a blue checkbox. At the bottom, there is an unchecked checkbox labeled 'Show all settings'.

Dataiku Application

Test the app

Our application is ready ! Don't forget to **save** your changes.

Let's **test** the application, to see how it works.



Dataiku Application

Test the app

Explore your test instance for your application. You should recognize the different tiles you have designed in the Application Designer !



Test instance for FINALADVDESIGNER

Test instance generated from app designer

[< Back to application](#)SWITCH TO PROJECT VIEW

Download Top 10 Merchants

Input the state you want, click run and download the dataset

Select State	State Name	Nothing selected ▼
Run	RUN NOW	
Download	DOWNLOAD	

Dataiku Application

Test the app

- Select a state from the dropdown menu (whichever one you like !)
- Run the scenario (and wait for it to finish)
- Then Download the output dataset, and check that it contains data for your selected state.



Test instance for FINALADVDESIGNER

Test instance generated from app designer

[< Back to application](#)SWITCH TO PROJECT VIEW

Download Top 10 Merchants

Input the state you want, click run and download the dataset

Select State	State Name	New Hampshire
Run	RUN NOW ⚠ warning Oct. 11 14:04:11 RUN DETAILS	
Download	DOWNLOAD	

Well done !

Your app can now be re-used by any colleague to generate those insights.

Dataiku Application

Additional information

- By default, applications are created such that end-users that want to instantiate the application need to have **Execute app** permission enabled
- To manage access, the application designer can configure the execute permissions for the application.

The screenshot displays the Dataiku Application Designer interface. The top section shows the 'Application designer' tab with a sidebar on the left containing 'Application header', 'Application features', 'Included content', 'Download Merchants', 'Upload New Merchants', 'Select State', 'Click to Run', and 'Download'. The main area shows the 'tim test' application, created by tim.yap on Jan 07th 2023. A dropdown menu for 'Users with execute permission' is open, showing 'Everyone' (Any user can create an instance of the app) and 'Users with execute permission' (Only users having 'execute app' permission on this project can instantiate the app).

The bottom section shows the 'Project security' settings for the 'tim test' project. The 'Permissions' tab is active, showing the following settings:

- Project owner: tim.yap
- Project visibility: Inherit global settings (Private)
- Project access requests: Inherit global settings (Disabled)

The permissions table lists various actions and their status for the 'space_administrators' group:

Name	Admin	Read project content	Write project content	Execute app	Publish on workspaces	Export datasets	Read dashboards	Write dashboards	Run scenarios	Manage authorized objects	Manage shared objects
space_administrators	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒

At the bottom, there are buttons to '+ GRANT ACCESS TO GROUP' and '+ GRANT ACCESS TO USER'.

More examples of Dataiku Application

Dataiku gallery

 Dataiku DSS

Upgrade Profile

Public Gallery

New

Recommended Content



Product tour
Get a tour of Dataiku
Explore Dataiku basics with a walkthrough of a prebuilt project.



Live demo
Join a Live Demo of Dataiku
Discover Dataiku with different interactive and biweekly live events — or on-demand.



Video
End to End Demo
A tour of Dataiku's end to end capabilities by exploring a real life use case around...



Documentation
Reference Documentation
The main documentation of the concepts, interfaces and features of Dataiku DSS.

Recent and favorite items

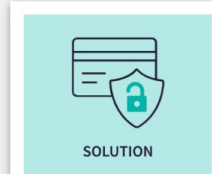
Applications [See all >](#)



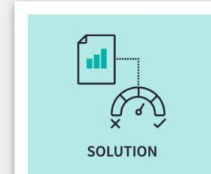
SOLUTION
Batch Performanc...



SOLUTION
Clinical Site Intelli...



SOLUTION
Credit Card Fraud



SOLUTION
Credit Scoring



SOLUTION
Customer Lifetime...



SOLUTION
Customer S...



Dataiku Application in Product Recommendations Solution

[Dataiku gallery](#)

 Application designer

ContentAdvanced **SAVED**TESTACTIONS

Application header
Application features
Included content
Section
Connection settings
Datasets connection informa
Product pictures connection
Reconfigure connection setti
Data specifications
interactions history
user metadata
item metadata
item pictures
Load product recommendati
Columns identification
interactions history
user metadata
item metadata
Edit project variables

**SOLUTION**

Product Recommendations

Use this App to create an end-to-end interactive project for recommending products to your customers. Read More on the [Knowledge Base](#)

- Click on the **CREATE APP INSTANCE** button to set up a new project.
- Select an existing project to consume its deliverables or continue working on it.

Instantiation allowed to Users with execute permission

Application visibility Inherit global settings (Private) When set to "Discoverable", users who do not have access to this application can still discover it and view its summary

Application execution permission requests Inherit global settings (Disabled) Allow users to request permission to execute this application

Application tags

 retail prepackaged-solution webapp **dataiku-application** + Add Asset + Add Technical Dimension + Add Business Function + Add Use-case Domain

Application instance features

Show flow section ☒

Show Lab section ☒

Show code section ☒

Show version control ☒

community.dataiku.com

- Public Q&A
- (like stack overflow)

doc.dataiku.com

- Extensive technical documentation about DSS

Additional Resources

academy.dataiku.com

- Tutorials
- Videos

gallery.dataiku.com

- Sample projects